



BENEMERITA UNIVERSIDAD AUTÓNOMA DE PUEBLA

INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

Periodo Primavera 2025

PROGRAMACION AVANZADA

FCCA-020

Profesor: Jaime Alejandro Romero Sierra



Práctica unidad 2

GESTIÓN DE PEDIDOS EN UNA CAFETERIA

Explicación del funcionamiento del sistema

Nombre	Matricula
Ortiz García Luz Marisol	202450809

Contenido

Módulos y bibliotecas que serán utilizados en el desarrollo del programa.....	3
Clase SplashScreen	4
Clase UserManager	7
Métodos de Inicialización y Persistencia.....	7
Gestión de Usuarios	8
Métodos de Autenticación y Seguridad.....	9
Método de Autorización.....	10
Métodos de Administración	10
Métodos de Consulta.....	11
Clase Inventario.....	12
Clase Producto	17
Clase CafeteriaApp.....	18

Módulos y bibliotecas que serán utilizados en el desarrollo del programa

```
import tkinter as tk
from tkinter import messagebox, simpledialog, ttk
from PIL import Image, ImageTk
import os
import json
from datetime import datetime
import hashlib
import uuid
```

1. Módulos de Tkinter (Interfaz Gráfica)

- **tkinter**: Librería principal para crear la interfaz gráfica (ventanas, botones, etc.). Se importa como tk para abreviar su uso (ej: tk.Button() en lugar de tkinter.Button()).
- **messagebox**: Para mostrar diálogos emergentes (alertas, confirmaciones).
- **simpledialog**: Para pedir información al usuario (como inputs simples).
- **ttk**: Versión mejorada de widgets de Tkinter (botones más modernos, tablas, etc.).

2. Procesamiento de Imágenes:

- **PIL** (Python Imaging Library): Permite cargar y manipular imágenes.
 - Image: Para abrir y redimensionar imágenes.
 - ImageTk: Convierte imágenes de PIL a formato compatible con Tkinter.

3. Sistema y Archivos:

- **os**: Para operaciones del sistema operativo (manejo de rutas de archivos, carpetas).
- **json**: Para guardar y cargar datos en formato JSON (usuarios, inventario, pedidos).

4. Funcionalidades Adicionales:

- **datetime**: Para manejar fechas/horas (ej: registrar cuándo se hizo un pedido).
- **hashlib**: Para encriptar contraseñas (con el método sha256).
- **uuid**: Genera IDs únicos (para usuarios, pedidos, etc.).

Clase SplashScreen



Pantalla de presentación del programa.

1. Metodo `_init_`: Inicialización

```
class SplashScreen:
    def __init__(self, root, main_app_class):
        self.root = root # Ventana principal de Tkinter
        (parameter) self: Self@SplashScreen # Clase principal que se ejecutará después
        self.root.title("Bienvenido")
        self.root.geometry("400x600") # Tamaño de la ventana
        self.root.configure(bg="white") # Fondo blanco

        try:
            self.root.iconbitmap("D:/DESCARGAS Y COSAS/cafe.ico") # Icono de la ventana
        except Exception as e:
            print(f"No se pudo cargar el icono: {e}")

        # Mostrar contenido del splash screen
        self.show_content()

        # Programa la transición después de 3 segundos
        self.root.after(3000, self.transition_to_main_app)
```

Funciones clave:

- Configura la ventana inicial (título, tamaño, color de fondo)
- Intenta cargar un icono personalizado (maneja errores si no existe)
- Llama a `show_content()` para mostrar los elementos visuales
- Programa la transición a la aplicación principal después de 3 segundos (`after(3000, ...)`)

2. Metodo `show_content`: Diseño Visual

Estructura visual:

- Frame principal: Contenedor base con fondo blanco
- Parte superior:
 - Intenta cargar una imagen de bienvenida (bienvenido1.png)
 - Si falla, muestra un texto alternativo "Bienvenido!!"
- Parte inferior:
 - Carga una segunda imagen (intro4.png) como logo
 - Si falla, muestra texto alternativo "Logo de la Cafetería"
- Usa try-except para manejar errores si las imágenes no existen
- Image.Resampling.LANCZOS proporciona alta calidad al redimensionar
- self.welcome_img_tk mantiene referencia a la imagen para evitar garbage collection

```
class SplashScreen:
    def show_content(self):
        # Frame principal (ocupa toda la ventana)
        main_frame = tk.Frame(self.root, bg="white")
        main_frame.pack(fill=tk.BOTH, expand=True)

        # Frame superior para el texto (ocupa el espacio mínimo necesario)
        try:
            # Ruta de la imagen que reemplazará al texto (ej: "bienvenida.png")
            welcome_img_path = os.path.join("D:", "DESCARGAS Y COSAS", "bienvenido1.png")
            welcome_img = Image.open(welcome_img_path)
            welcome_img = welcome_img.resize((350, 200), Image.Resampling.LANCZOS) # Ajusta el tamaño según necesites
            self.welcome_img_tk = ImageTk.PhotoImage(welcome_img)

            # Mostrar la imagen (en lugar del Label de texto)
            welcome_label = tk.Label(
                main_frame,
                image=self.welcome_img_tk,
                bg="white"
            )
            welcome_label.pack(pady=(100, 20)) # Margen superior de 100px, inferior de 20px

        except Exception as e:
            print(f"Error al cargar imagen de bienvenida: {e}")
            # Si falla, muestra un texto alternativo (opcional)
            tk.Label(
                main_frame,
                text="Bienvenido!!",
                font=("Arial", 18, "bold"),
                bg="white"
            ).pack(pady=(100, 5))

        # Frame inferior para la imagen (se expandirá hacia abajo)
        bottom_frame = tk.Frame(main_frame, bg="white")
        bottom_frame.pack(fill=tk.BOTH, expand=True) # Ocupa todo el espacio restante

        # Carga y muestra la imagen EN LA PARTE INFERIOR
        try:
            img_path = os.path.join("D:", "DESCARGAS Y COSAS", "intro4.png")
            img = Image.open(img_path)
            img = img.resize((300, 350), Image.Resampling.LANCZOS)
            self.img_tk = ImageTk.PhotoImage(img)

            # Contenedor para centrar la imagen en el frame inferior
            img_container = tk.Frame(bottom_frame, bg="white")
            img_container.pack(expand=True, fill=tk.BOTH)
```

```

img_label = tk.Label(
    img_container,
    image=self.img_tk,
    bg="white"
)
img_label.pack(side=tk.BOTTOM, pady=(0)) # Pegado al fondo con margen inferior
except Exception as e:
    print(f"Error al cargar imagen de bienvenida: {e}")
    tk.Label(bottom_frame, text="Logo de la Cafetería", bg="white").pack(side=tk.BOTTOM)

```

3. Método `transition_to_main_app`: Cambio a App Principal

```

class SplashScreen:

    def transition_to_main_app(self):
        # Destruye la ventana actual
        self.root.destroy()

        # Crea la ventana principal y la aplicación
        root = tk.Tk()
        self.main_app_class(root)
        root.mainloop()

```

Flujo de ejecución:

- Destruye la ventana actual del splash screen
- Crea una nueva ventana raíz de Tkinter
- Instancia la clase principal (CafeteriaApp) pasando la nueva ventana
- Inicia el bucle principal de la interfaz gráfica
- Relación con el Resto del Sistema

Clase UserManager

Clase de gestión de usuarios del programa.

1. Método `__init__`:

Es el constructor de la clase UserManager y se ejecuta automáticamente al crear una nueva instancia.

```
class UserManager:
    def __init__(self):
        self.users_file = os.path.join("D:", "DESCARGAS Y COSAS", "users.json")
        self.current_user = None # Usuario actualmente Logueado
        self.users = self.load_users() # Carga usuarios desde JSON

        # Crear usuario admin por defecto si no existe
        if not self.users:
            self.create_default_admin()
```

Estructura del JSON

```
self.users_file = os.path.join("D:", "DESCARGAS Y COSAS", "users.json")
```

- **Ruta absoluta:** Apunta a una ubicación específica en disco (D:/DESCARGAS Y COSAS/)
- **Formato JSON:** Almacena un array de objetos de usuario
- **Ejemplo real:**

```
D: > DESCARGAS Y COSAS > users.json > ...
1  [
2      {
3          "id": "a31752bb-2bd4-49fa-b7f2-a96e5648a649",
4          "username": "admin",
5          "password_hash": "5640f504804c5c11fec8d030ae83cb40a7854f7904f0121aaa9489be70fd3c90",
6          "is_owner": true,
7          "created_at": "2025-04-14 21:02:28"
8      }
9  ]
```

Métodos de Inicialización y Persistencia

2. Método `load_users()`:

```
def load_users(self):
    # Carga los usuarios desde el archivo JSON
    try:
        with open(self.users_file, 'r') as f:
            return json.load(f)
    except (FileNotFoundError, json.JSONDecodeError):
        return []
```

Función: Carga usuarios desde el archivo JSON

Flujo:

- Intenta abrir el archivo en modo lectura
- Si existe y es válido, carga el contenido con `json.load()`
- Si el archivo no existe o está corrupto, retorna lista vacía

3. Método `save_users()`:

```
def save_users(self):
    # Guarda los usuarios en el archivo JSON
    try:
        with open(self.users_file, 'w') as f:
            json.dump(self.users, f, indent=4)
    except Exception as e:
        messagebox.showerror("Error", f"No se pudieron guardar los usuarios: {str(e)}")
```

Función: Guarda los usuarios en disco

Detalles clave:

- `indent=4` formatea el JSON para legibilidad
- Manejo de errores muestra mensaje gráfico

Frecuencia de uso: Ocurre tras cada modificación importante

Gestión de Usuarios

4. Método `create_default_admin()`:

```
def create_default_admin(self):
    # Crea un usuario administrador por defecto
    default_user = {
        "id": str(uuid.uuid4()), # Crea un identificador universal único
        "username": "admin", # Credenciales de Acceso
        "password_hash": self.hash_password("admin123"), # Hash de Seguridad
        "is_owner": True, # Otorga permisos máximos
        "created_at": datetime.now().strftime("%Y-%m-%d %H:%M:%S") # captura la fecha/hora exacta de creación
    }
    self.users.append(default_user) # Añade a la lista en memoria
    self.save_users() # Guarda permanentemente en el JSON
```

Propósito: Crea un usuario administrador predeterminado cuando la aplicación se ejecuta por primera vez (o cuando no hay usuarios registrados).

Cuando se ejecuta: Automáticamente desde `__init__()` si `load_users()` retorna una lista vacía.

Datos generados:

- UUID único
- Hash de "admin123"
- Timestamp exacto

Persistencia: Guarda inmediatamente en disco

5. Método `add_user()`:

```
def add_user(self, username, password, is_admin=False):
    # Agrega un nuevo usuario
    # Verificar si el usuario ya existe
    if any(user["username"] == username for user in self.users):
        raise ValueError("El nombre de usuario ya existe")
    # Creación del Objeto Usuario
    new_user = {
        "id": str(uuid.uuid4()),
        "username": username,
        "password_hash": self.hash_password(password),
        "is_admin": is_admin,
        "created_at": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    }

    self.users.append(new_user)
    self.save_users()
```

Propósito: Registrar nuevos usuarios en el sistema de manera segura

Persistencia: Auto-guardado

Parámetros:

- username: Nombre de usuario (debe ser único)
- password: Contraseña en texto plano (se hashea antes de almacenar)
- is_admin: Booleano que determina si tendrá privilegios administrativos (default: False)

Métodos de Autenticación y Seguridad

6. Método `hash_password()`:

Función: Hashea una contraseña usando SHA-256 con salt

```
def hash_password(self, password):
    salt = "cafeteria_salt"
    return hashlib.sha256((password + salt).encode()).hexdigest()
```

7. Método authenticate():

Autentica un usuario

```
def authenticate(self, username, password):
    for user in self.users:
        if user["username"] == username and user["password_hash"] == self.hash_password(password):
            self.current_user = user
            return True
    return False
```

Seguridad: Nunca revela qué falló (usuario o contraseña)

Método de Autorización

8. Método is_owner():

Función: Verifica si el usuario actual es el dueño

```
def is_owner(self):
    return self.current_user and self.current_user.get("is_owner", False)
```

Seguridad: Doble verificación (current_user existe y es owner)

Métodos de Administración

9. Método remove_user():

Función: Elimina un usuario

```
def remove_user(self, username):
    if not self.is_owner():
        raise PermissionError("Solo el dueño puede eliminar usuarios")

    if username == self.current_user["username"]:
        raise ValueError("No puedes eliminarte a ti mismo")

    self.users = [user for user in self.users if user["username"] != username]
    self.save_users()
```

Protecciones:

- Solo dueño puede eliminar
- No auto-eliminación

Técnica: Filtrado con list comprehension

10. Método `change_password()`:

Función: Cambia la contraseña de un usuario

```
def change_password(self, username, new_password):

    if not self.is_owner() and username != self.current_user["username"]:
        raise PermissionError("Solo puedes cambiar tu propia contraseña")

    for user in self.users:
        if user["username"] == username:
            user["password_hash"] = self.hash_password(new_password)
            self.save_users()
            return True
    return False
```

Reglas:

- Dueño: Cambia cualquier contraseña
- Usuarios: Solo la propia

Persistencia: Guardado inmediato

Métodos de Consulta**11. Método `get_all_users()`:**

Función: Obtiene todos los usuarios (sin información sensible).

```
def get_all_users(self):

    return [{
        "username": user["username"],
        "is_admin": user.get("is_admin", False),
        "is_owner": user.get("is_owner", False)
    } for user in self.users]
```

Seguridad: No expone información sensible (hashes, IDs)

Uso típico: Para mostrar listados de usuarios

Clase Inventario

Es el sistema de gestión de stock de ingredientes para la cafetería.

1. Método `__init__`:

Constructor

```
class Inventario:
    def __init__(self):
        # Ingredientes predeterminados
        self.ingredientes = {
            "leche": {"cantidad": 100, "unidad": "ml"},
            "harina": {"cantidad": 100, "unidad": "gr"},
            "café": {"cantidad": 100, "unidad": "gr"},
            "chocolate": {"cantidad": 100, "unidad": "gr"},
            "azúcar": {"cantidad": 100, "unidad": "gr"},
            "huevos": {"cantidad": 100, "unidad": "unidades"},
            "agua": {"cantidad": 100, "unidad": "ml"}
        }
        self.ruta_inventario = os.path.join("D:", "DESCARGAS Y COSAS", "inventario.json")
        self.cargar_inventario() # ¡Aquí se genera el archivo si no existe!
```

Funcionamiento detallado:

Inicialización de datos predeterminados:

- Crea un diccionario con 7 ingredientes básicos
- Cada ingrediente tiene:
 - **cantidad:** Valor inicial de 100 unidades
 - **unidad:** "ml" para líquidos, "gr" para sólidos, "unidades" para contables

Configuración de rutas:

- Establece la ruta absoluta al archivo JSON de inventario
- Usa `os.path.join()` para compatibilidad entre sistemas operativos

Carga inicial:

- Llama a `cargar_inventario()` para intentar cargar datos existentes

2. Método `cargar_inventario()`:

```
def cargar_inventario(self):
    try:
        with open(self.ruta_inventario, 'r') as f:
            self.ingredientes = json.load(f)
    except FileNotFoundError:
        self.guardar_inventario()
    except json.JSONDecodeError:
        messagebox.showerror("Error", "El archivo de inventario está corrupto. Se creará uno nuevo.")
        self.guardar_inventario() # Regenera con valores predeterminados
```

Función: Carga datos desde JSON o inicializa uno nuevo

Manejo de errores:

- **Archivo no encontrado** → Llama a `guardar_inventario()` para crear uno nuevo con valores predeterminados
- **JSON corrupto** → Notifica (Muestra mensaje de error) y regenera el archivo con valores predeterminados

El inventario.json tendrá este formato:

```
{
  "leche": {
    "cantidad": 100,
    "unidad": "ml"
  },
  "harina": {
    "cantidad": 100,
    "unidad": "gr"
  },
  "café": {
    "cantidad": 100,
    "unidad": "gr"
  }
  // ... otros ingredientes
}
```

3. Método `guardar_inventario()`:

Función: Guarda el inventario actual en el archivo JSON y se llama automáticamente después de cada modificación al inventario.

```
def guardar_inventario(self):
    try:
        with open(self.ruta_inventario, 'w') as f:
            json.dump(self.ingredientes, f, indent=4, ensure_ascii=False)
    except Exception as e:
        messagebox.showerror("Error", f"No se pudo guardar el inventario: {str(e)}")
```

Manejo de errores:

- Captura cualquier excepción durante el guardado
- Muestra mensaje de error con detalles

4. Método `verificar_disponibilidad()`:

Función: Control de stock, verifica que el ingrediente esté registrado y si hay suficiente cantidad (sea mayor o igual a lo requerido) de un ingrediente.

```
def verificar_disponibilidad(self, ingrediente, cantidad):
    return ingrediente in self.ingredientes and self.ingredientes[ingrediente]["cantidad"] >= cantidad
```

5. Método consumir_ingredientes():

Función: Gestión de pedidos, consume una cantidad de ingredientes del inventario y lanza ValueError si no hay suficiente cantidad.

```
def consumir_ingredientes(self, ingredientes):
    # Primero verificar disponibilidad
    for ingrediente, cantidad in ingredientes.items():
        if not self.verificar_disponibilidad(ingrediente, cantidad):
            raise ValueError(f"No hay suficiente {ingrediente}")

    # Luego consumir
    for ingrediente, cantidad in ingredientes.items():
        self.ingredientes[ingrediente]["cantidad"] -= cantidad
    self.guardar_inventario()
```

Fase de actualización:

- Reduce las cantidades de cada ingrediente
- Guarda automáticamente los cambios

6. Método agregar_ingrediente():

Función: Añade cantidad a un ingrediente existente y no permite crear nuevos ingredientes (para eso está agregar_nuevo_ingrediente()).

```
def agregar_ingrediente(self, ingrediente, cantidad):
    if ingrediente in self.ingredientes:
        self.ingredientes[ingrediente]["cantidad"] += cantidad
    else:
        raise KeyError(f"Ingrediente {ingrediente} no existe")
    self.guardar_inventario()
```

7. Método actualizar_ingrediente():

Función: Actualiza la cantidad y/o unidad de un ingrediente y si el ingrediente no existe, lo crea.

```
def actualizar_ingrediente(self, ingrediente, cantidad, unidad=None):
    if ingrediente in self.ingredientes:
        # Actualización existente
        self.ingredientes[ingrediente]["cantidad"] = cantidad
        if unidad:
            self.ingredientes[ingrediente]["unidad"] = unidad
    else:
        # Creación nueva
        if not unidad:
            unidad = self._determinar_unidad_predeterminada(ingrediente)
        self.ingredientes[ingrediente] = {"cantidad": cantidad, "unidad": unidad}
    self.guardar_inventario()
```

8. Método eliminar_ingrediente():

Función: Elimina un ingrediente del inventario

```
def eliminar_ingrediente(self, ingrediente):
    if ingrediente in self.ingredientes:
        del self.ingredientes[ingrediente]
        self.guardar_inventario()
        return True
    return False
```

9. Método agregar_nuevo_ingrediente():

Función: Agrega un nuevo ingrediente al inventario

```
def agregar_nuevo_ingrediente(self, nombre, cantidad, unidad):
    if nombre in self.ingredientes:
        raise ValueError(f"Ingrediente {nombre} ya existe")

    if not unidad:
        unidad = self._determinar_unidad_predeterminada(nombre)

    self.ingredientes[nombre] = {
        "cantidad": cantidad,
        "unidad": unidad
    }
    self.guardar_inventario()
```

10. Método _determinar_unidad_predeterminada():

Función: Determina la unidad predeterminada basada en el nombre del ingrediente.

```
def _determinar_unidad_predeterminada(self, ingrediente):
    if ingrediente in ["leche", "agua", "jugo"]:
        return "ml"
    elif ingrediente in ["huevos", "unidades", "piezas"]:
        return "unidades"
    return "gr"
```

11. Método obtener_estado():

Función: Devuelve una copia del estado actual del inventario

Propósito:

- Proporciona una copia segura del inventario
- Evita que modificaciones externas afecten los datos internos

```
def obtener_estado(self):
    return self.ingredientes.copy()
```

12. Método obtener_ingrediente():

Función: Obtiene los datos de un ingrediente específico

Comportamiento:

- Retorna None si el ingrediente no existe
- Útil para verificaciones antes de operaciones

```
def obtener_ingrediente(self, nombre):
    return self.ingredientes.get(nombre, None)
```

13. Método reiniciar_inventario():

Función: Reinicia el inventario a los valores predeterminados y se guardan en el JSON

```
def reiniciar_inventario(self):
    self.ingredientes = {
        "leche": {"cantidad": 100, "unidad": "ml"},
        "harina": {"cantidad": 100, "unidad": "gr"},
        "café": {"cantidad": 100, "unidad": "gr"},
        "chocolate": {"cantidad": 100, "unidad": "gr"},
        "azúcar": {"cantidad": 100, "unidad": "gr"},
        "huevos": {"cantidad": 100, "unidad": "unidades"},
        "agua": {"cantidad": 100, "unidad": "ml"}
    }
    self.guardar_inventario()
```

Uso típico:

- En casos de corrupción de datos
- Para pruebas y desarrollo
- Como función de emergencia

Clase Producto

Es el modelo de datos que representa los artículos disponibles en la cafetería (bebidas, postres, etc.).

1. Método `__init__`:

Constructor

Propósito:

- Crear instancias de productos con toda la información necesaria para:
- Mostrarlos en el menú
- Calcular costos
- Gestionar el inventario

```
class Producto:
    def __init__(self, nombre, precio, categoria, imagen_path, ingredientes=None):
        self.nombre = nombre
        self.precio = precio
        self.categoria = categoria
        self.imagen_path = imagen_path
        self.ingredientes = ingredientes or {}
```

Relaciones clave:

- Con Inventario: Los ingredientes se consumen del inventario al preparar el producto
- Con CafeteriaApp: Se usan para mostrar el menú y procesar pedidos

2. Método `__str__`():

Función: Representación legible del producto cuando se convierte a string (para interfaces, debugging, etc.)

```
def __str__(self):
    return f"{self.nombre} (${self.precio:.2f})"
```

Clase CafeteriaApp

Es la clase principal de la aplicación, que se encarga de gestionar la interfaz gráfica (con tkinter) y la lógica de operación para usuarios, pedidos, inventario y productos de una cafetería.

1. Método `__init__`:

Constructor

Propósito: Configurar todo lo necesario para iniciar la aplicación gráfica de la cafetería, incluyendo:

- Configura la ventana principal
- Inicializa todos los componentes esenciales (gestión de usuarios, inventario)
- Carga los datos persistentes
- Prepara el sistema para mostrar interfaces gráficas
- Muestra la pantalla de inicio y asigna un ícono a la ventana.

```
class CafeteriaApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Gestión de Cafetería")
        self.root.geometry("400x600")
        self.root.configure(bg="white")

        try:
            self.root.iconbitmap("D:/DESCARGAS Y COSAS/cafe.ico")
        except Exception as e:
            print(f"No se pudo cargar el icono: {e}") # Manejo de error si el archivo no existe

        self.user_manager = UserManager()
        self.inventario = Inventario()
        self.pedidos = []
        self.cargar_pedidos()
        self.cargar_productos()

        self.imagenes_tk = []
        self.mostrar_pantalla_inicio()
```

2. Método `mostrar_pantalla_inicio()`:

Función: Se encarga de mostrar la pantalla inicial del programa, con una interfaz gráfica que permite al usuario seleccionar si desea ingresar como cliente o como administrador.

```

def mostrar_pantalla_inicio(self):
    # Limpia la pantalla actual eliminando todos los widgets existentes
    self.limpiar_pantalla()

    # Crea un frame principal que actuará como contenedor para todos los demás elementos
    main_frame = tk.Frame(self.root, bg="white")
    main_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

    # Frame especial para contener las imágenes, centrado en la pantalla
    images_frame = tk.Frame(main_frame, bg="white")
    images_frame.pack(pady=(0, 30))

    # Intenta cargar y mostrar la imagen de inicio
    try:
        # Imagen
        img2_path = os.path.join("D:", "DESCARGAS Y COSAS", "imageninicio.png") # Ruta
        img2 = Image.open(img2_path)
        img2 = img2.resize((280, 360), Image.Resampling.LANCZOS)
        self.img2_tk = ImageTk.PhotoImage(img2)
        img2_label = tk.Label(images_frame, image=self.img2_tk, bg="white")
        img2_label.pack()
    except Exception as e:
        print(f"Error al cargar imágenes: {e}")
        # Si hay error, muestra un mensaje en lugar de las imágenes
        tk.Label(images_frame, text="Logotipos no disponibles", bg="#f5f5f5").pack()

    # Crea y muestra un texto indicativo
    tk.Label(
        main_frame,
        text="Ingresa como:",
        font=("Arial", 10),
        bg="white"
    ).pack(pady=(0, 5))

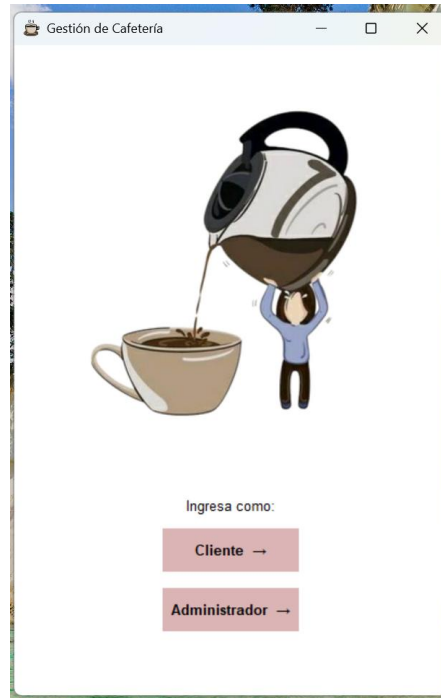
    # Crea un frame especial para contener los botones de opción
    buttons_frame = tk.Frame(main_frame, bg="white")
    buttons_frame.pack()

    # Define un estilo común para ambos botones
    button_style = {
        "width": 15,
        "height": 2,
        "font": ("Arial", 10, "bold"),
        "bg": "#D8B5B5",
        "fg": "Black",
        "border": 0
    }

    # Crea y muestra los botones para ingresar como cliente y administrador
    tk.Button(
        buttons_frame,
        text="Cliente →",
        command=self.iniciar_como_cliente,
        **button_style
    ).pack(pady=5)

    tk.Button(
        buttons_frame,
        text="Administrador →",
        command=self.iniciar_como_admin,
        **button_style
    ).pack(pady=10)

```



2. Método `limpiar_pantalla()`:

Funciones:

- Eliminación de componentes gráficos:
Elimina todos los widgets (botones, labels, frames, etc.) que estén contenidos directamente en la ventana principal (`self.root`).
- Preparación para nueva interfaz:
Limpia completamente la interfaz actual para poder construir una nueva pantalla desde cero.
- Gestión de memoria:
Al destruir los widgets, libera recursos del sistema que estaban siendo usados por esos elementos gráficos.

```
def limpiar_pantalla(self):  
    for widget in self.root.winfo_children():  
        widget.destroy()
```

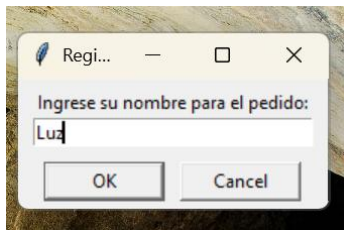
3. Método `iniciar_como_cliente()`:

Funciones:

- Recolección de datos del usuario: Solicita al cliente que ingrese su nombre mediante un cuadro de diálogo estándar.

- Validación básica: Verifica que se haya ingresado un nombre (no vacío) antes de continuar.
- Gestión de estado: Almacena el nombre del cliente como variable de instancia (`self.nombre_cliente`) para uso posterior.
- Transición de pantallas: Inicia la interfaz específica para clientes llamando a `mostrar_interfaz_cliente()`.

```
def iniciar_como_cliente(self):
    nombre_cliente = simpledialog.askstring("Registro", "Ingrese su nombre para el pedido:")
    if nombre_cliente:
        self.nombre_cliente = nombre_cliente
        self.mostrar_interfaz_cliente(nombre_cliente)
```



4. Método `iniciar_como_admin()`:

```
def iniciar_como_admin(self):
    if not self.user_manager.users: # Si no hay usuarios, crear admin por defecto
        self.user_manager.create_default_admin()

    self.mostrar_login_admin()
```

Funciones:

- Comprueba si hay usuarios registrados en el sistema (`self.user_manager.users`).
 - Si no hay usuarios (`if not self.user_manager.users`), llama a `create_default_admin()` para crear un usuario administrador predeterminado.
 - Caso 1: No hay usuarios → Crea admin por defecto → Muestra login
- Cuando no hay usuarios registrados (archivo JSON vacío o inexistente), realiza estas acciones en el JSON:

```
{
  "id": "a31752bb-2bd4-49fa-b7f2-a96e5648a649",
  "username": "admin",
  "password_hash": "5640f504804c5c11fec8d030ae83cb40a7854f7904f0121aaa9489be70fd3c90",
  "is_owner": true,
  "created_at": "2025-04-14 21:02:28"
}
```

- Caso 2: Hay usuarios existentes → Muestra login directamente

- Muestra la interfaz de login para administradores llamando a `mostrar_login_admin()`.

5. Método `mostrar_login_admin()`:

Funcion: Se encarga de mostrar la pantalla de inicio de sesión para administradores. Su objetivo principal es proporcionar una interfaz donde los usuarios con permisos administrativos puedan autenticarse antes de acceder a las funciones restringidas del sistema.

El método crea el formulario de autenticación:

Campo "Usuario": Entrada de texto para el nombre de usuario (`self.username_entry`).

Campo "Contraseña": Entrada de texto que oculta caracteres con * (`self.password_entry`).

```
def mostrar_login_admin(self):
    self.limpiar_pantalla()

    main_frame = tk.Frame(self.root, bg="white")
    main_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

    # Imagen de Login
    try:
        login_img_path = os.path.join("D:", "DESCARGAS Y COSAS", "admin_login.png")
        login_img = Image.open(login_img_path)
        login_img = login_img.resize((200, 150), Image.Resampling.LANCZOS)
        self.login_img_tk = ImageTk.PhotoImage(login_img)
        tk.Label(main_frame, image=self.login_img_tk, bg="white").pack(pady=10)
    except Exception as e:
        print(f"Error al cargar imagen de login: {e}")
        tk.Label(main_frame, text="Acceso Administrativo", font=("Arial", 16, "bold"), bg="white").pack(pady=10)

    # Campos de Login
    tk.Label(main_frame, text="Usuario:", bg="white").pack(pady=5)
    self.username_entry = tk.Entry(main_frame)
    self.username_entry.pack(pady=5)

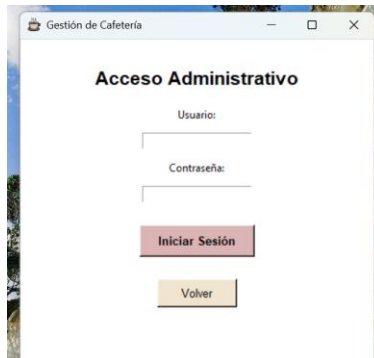
    tk.Label(main_frame, text="Contraseña:", bg="white").pack(pady=5)
    self.password_entry = tk.Entry(main_frame, show="*")
    self.password_entry.pack(pady=5)

    # Botón de Login
    tk.Button(
        main_frame,
        text="Iniciar Sesión",
        command=self.verificar_login_admin,
        bg="#D8B5B5", fg="black", font=("Arial", 10, "bold"), pady=5, width=15
    ).pack(pady=20)

    # Botón para volver
    tk.Button(
        main_frame,
        text="Volver",
        command=self.mostrar_pantalla_inicio,
        bg="#F1E5D1", fg="black", font=("Arial", 10), pady=3, width=10
    ).pack(pady=5)
```

La interfaz cuenta con 2 botones de acción:

- Iniciar Sesión: Al hacer clic, ejecuta `self.verificar_login_admin()` para validar credenciales.
- Volver: Regresa a la pantalla inicial (`self.mostrar_pantalla_inicio()`).



6. Método `verificar_login_admin()`:

Funcion: Valida las credenciales de acceso ingresadas por un usuario que intenta acceder como administrador. Su función principal es autenticar al usuario y, si es exitoso, permitirle acceder al panel de administración.

```
def verificar_login_admin(self):
    # Recolección de credenciales
    username = self.username_entry.get()
    password = self.password_entry.get()

    # Validación de campos vacíos, si alguno falta, muestra un mensaje de error y detiene el proceso.
    if not username or not password:
        messagebox.showerror("Error", "Usuario y contraseña son requeridos")
        return

    # Autenticación del usuario para verificar si las credenciales son correctas.
    if self.user_manager.authenticate(username, password):
        self.mostrar_interfaz_admin() # Credenciales correctas
    else:
        # Credenciales incorrectas
        messagebox.showerror("Error", "Usuario o contraseña incorrectos")
```

7. Método `mostrar_interfaz_cliente()`:

Funcion: Muestra la interfaz gráfica para clientes después de que ingresan su nombre, con opciones para realizar pedidos o volver al inicio.

```
def mostrar_interfaz_cliente(self, nombre_cliente):
    # Limpia la pantalla actual para preparar la nueva interfaz
    self.limpiar_pantalla()

    # Crea el frame principal
    main_frame = tk.Frame(self.root, bg="white")
    main_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

    # Bloque try-except para cargar y mostrar la imagen de la ventana de cliente
    try:
        # Ruta completa a la imagen del cliente
        img_path = os.path.join("D:", "DESCARGAS Y COSAS", "cliente.png")
        # Abre y redimensiona la imagen
        img = Image.open(img_path)
        img = img.resize((320, 300), Image.Resampling.LANCZOS)
        self.cliente_header_img = ImageTk.PhotoImage(img) # Guardar referencia como atributo

        # Crea y muestra el Label con la imagen
        img_label = tk.Label(main_frame, image=self.cliente_header_img, bg="white")
        img_label.pack(pady=(0, 50))

    # Manejo de error si falla la carga de imagen
    except Exception as e:
        print(f"Error al cargar imagen: {e}")
        tk.Label(main_frame, text="Bienvenido", font=("Arial", 16, "bold"), bg="white").pack(pady=10)

    # Muestra un saludo personalizado con el nombre del cliente
    tk.Label(
        main_frame,
        text=f"Hola!, {nombre_cliente}",
        font=("Arial", 16, "bold"),
        bg="white"
    ).pack(pady=(0, 20))

    # Crea un frame contenedor para los botones
    buttons_frame = tk.Frame(main_frame, bg="white")
    buttons_frame.pack()

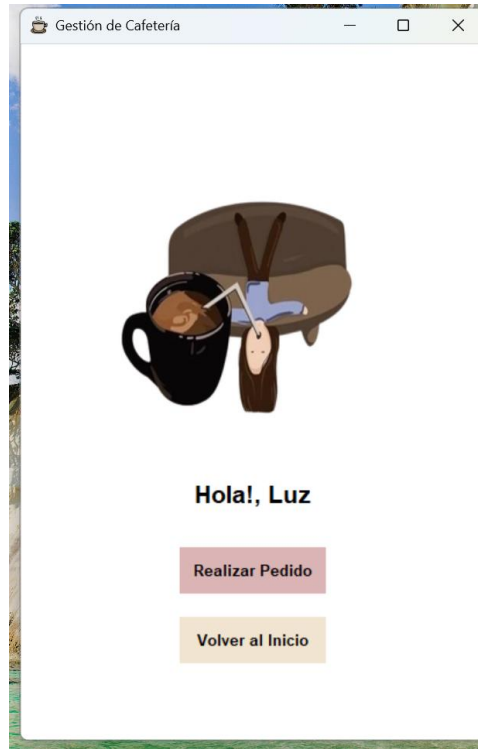
    # Define un estilo común para los botones
    button_style = {
        "width": 15,
        "height": 2,
        "font": ("Arial", 10, "bold"),
        "bg": "#DBB5B5",
        "fg": "Black",
        "border": 0
    }

    # Botones
    tk.Button(
        buttons_frame,
        text="Realizar Pedido",
        command=self.realizar_pedido,
        **button_style
    ).pack(pady=10, fill=tk.X)

    tk.Button(
        buttons_frame,
        text="Volver al Inicio",
        command=self.mostrar_pantalla_inicio,
        **{**button_style, "bg": "#F1E5D1"}
    ).pack(pady=10, fill=tk.X)
```

Relación con otros componentes:

- Se llama desde: `iniciar_como_cliente()` después de obtener el nombre del usuario
- Llama a:
 - `realizar_pedido()` para comenzar el proceso de compra
 - `mostrar_pantalla_inicio()` para regresar al menú principal
- Depende de: La imagen `cliente.png` (pero tiene alternativa si no existe)

**8. Método `mostrar_interfaz_admin()`:**

Funcion: Muestra la interfaz gráfica del panel de administración de la cafetería, sirve como menú principal para que los administradores accedan a todas las funciones de gestión del sistema.

Barra de usuario:

- Muestra información del usuario logueado
- Botón para cerrar sesión (vuelve a pantalla inicial)
- Si es owner/gerente, muestra botón adicional para gestión de usuarios

Botones de funciones:

- Agregar Producto: Abre formulario para nuevos productos

- Ver Productos: Muestra lista de productos existentes
- Ver Inventario: Muestra estado de ingredientes
- Historial de Ventas: Muestra registro de pedidos

```
def mostrar_interfaz_admin(self):
    # Limpia la pantalla actual eliminando widgets anteriores
    self.limpiar_pantalla()

    # Crea y configura el frame principal
    main_frame = tk.Frame(self.root, bg="white")
    main_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

    # Barra superior con info de usuario
    user_frame = tk.Frame(main_frame, bg="#f5f5f5", padx=10, pady=5)
    user_frame.pack(fill=tk.X, pady=(0, 20))

    current_user = self.user_manager.current_user["username"]
    is_owner = self.user_manager.is_owner()

    # Etiqueta con nombre de usuario y rol
    tk.Label(
        user_frame,
        text=f"Hola!, {current_user} {'(Gerente)' if is_owner else ''}",
        bg="#f5f5f5",
        font=("Arial", 10, "bold")
    ).pack(side=tk.LEFT)

    # Botón para cerrar sesión
    tk.Button(
        user_frame,
        text="Cerrar Sesión",
        command=self.mostrar_pantalla_inicio,
        bg="#F1E5D1",
        fg="black",
        font=("Arial", 8),
        padx=5
    ).pack(side=tk.RIGHT)

    # Botón adicional para gestión de usuarios (solo visible para owners)
    if is_owner:
        tk.Button(
            user_frame,
            text="Gestionar Usu.",
            command=self.mostrar_gestion_usuarios,
            bg="#DBB5B5",
            fg="black",
            font=("Arial", 8),
            padx=5
        ).pack(side=tk.RIGHT, padx=5)
```

```

#Imagen decorativa del panel de administración
try:
    # Intenta cargar y mostrar una imagen específica
    admin_img_path = os.path.join("D:", "DESCARGAS Y COSAS", "administrador.png")
    admin_img = Image.open(admin_img_path)
    admin_img = admin_img.resize((150, 190), Image.Resampling.LANCZOS)
    self.admin_header_img = ImageTk.PhotoImage(admin_img) # Guardar referencia

    img_label = tk.Label(main_frame, image=self.admin_header_img, bg="white")
    img_label.pack(pady=(0, 20))

    # Si falla la carga de la imagen, muestra un texto alternativo
except Exception as e:
    print(f"Error al cargar imagen de admin: {e}")
    tk.Label(main_frame, text="Panel de Administración",
            font=("Arial", 18, "bold"), bg="white").pack(pady=10)

# Título principal del panel
tk.Label(
    main_frame,
    text="Panel de Administración",
    font=("Arial", 16, "bold"),
    bg="white"
).pack(pady=(0, 10))

# Crea y configura el frame para los botones de acciones administrativas
buttons_frame = tk.Frame(main_frame, bg="white")
buttons_frame.pack()

# Estilo común para los botones
button_style = {
    "width": 15,
    "height": 2,
    "font": ("Arial", 10, "bold"),
    "bg": "#DBB5B5",
    "fg": "Black",
    "border": 0
}

# Crea los botones para las diferentes funciones administrativas:
admin_buttons = [
    ("Agregar Producto", self.agregar_producto),
    ("Ver Productos", self.mostrar_inventario_productos),
    ("Ver Inventario", self.ver_inventario),
    ("Historial de Ventas", self.mostrar_historial_ventas)
]

# Crea cada botón con su función correspondiente
for text, command in admin_buttons:
    bg_color = "#F1E5D1" if text == "Volver al Inicio" else "#DBB5B5"
    tk.Button(
        buttons_frame,
        text=text,
        command=command,
        **{**button_style, "bg": bg_color}
    ).pack(pady=5, fill=tk.X)

```



9. Método `mostrar_gestion_usuarios()`:

Función: Muestra una ventana de diálogo para gestionar usuarios administradores del sistema. Permite agregar, eliminar y cambiar contraseñas de usuarios. Este método centraliza todas las operaciones de gestión de usuarios en una interfaz intuitiva y organizada.

Botones de acción:

- Agregar Usuario: Abre diálogo para crear nuevos usuarios
- Eliminar Usuario: Elimina el usuario seleccionado (con confirmación)
- Cambiar Contraseña: Permite modificar contraseñas (propia o de otros si es dueño)
- Cerrar: Cierra la ventana de gestión

```

def mostrar_gestion_usuarios(self):
    # Muestra la interfaz para gestionar usuarios
    dialog = tk.Toplevel(self.root)
    dialog.title("Gestión de Usuarios")
    dialog.geometry("600x400")

    # Frame principal que contendrá todos los elementos
    main_frame = tk.Frame(dialog, padx=10, pady=10)
    main_frame.pack(fill=tk.BOTH, expand=True)

    # Título de la ventana
    tk.Label(
        main_frame,
        text="Gestión de Usuarios Administradores",
        font=("Arial", 14, "bold")
    ).pack(pady=10)

    # Frame para la lista de usuarios
    list_frame = tk.Frame(main_frame)
    list_frame.pack(fill=tk.BOTH, expand=True)

    # Configuración del Treeview para mostrar usuarios
    columns = ("Usuario", "Rol") # Columnas a mostrar
    tree = ttk.Treeview(
        list_frame,
        columns=columns,
        selectmode="browse" # Modo de selección simple
    )

    # Configuración de columnas:
    tree.heading("#0", text="ID")
    tree.column("#0", width=0, stretch=tk.NO) # Ocultar columna ID

    # Configura encabezados y ancho de columnas
    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, width=150 if col == "Usuario" else 100)

    # Llenado del Treeview con datos de usuarios
    for i, user in enumerate(self.user_manager.get_all_users(), 1):
        rol = "Dueño" if user.get("is_owner") else "Admin" if user.get("is_admin") else "Usuario"
        tree.insert("", tk.END, text=str(i), values=(user["username"], rol))

    # Inserta fila en el Treeview
    tree.pack(fill=tk.BOTH, expand=True)

    # Frame para los botones de acción
    button_frame = tk.Frame(main_frame)
    button_frame.pack(fill=tk.X, pady=10)

    # Botones de acciones:

    tk.Button(
        button_frame,
        text="Agregar Usuario",
        command=lambda: self.agregar_usuario_dialog(dialog),
        bg="#3C5B6F",
        fg="white",
        font=("Arial", 10, "bold")
    ).pack(side=tk.LEFT, padx=5)

```

```

tk.Button(
    button_frame,
    text="Eliminar Usuario",
    command=lambda: self.eliminar_usuario(tree, dialog),
    bg="#E98580",
    fg="black",
    font=("Arial", 10, "bold")
).pack(side=tk.LEFT, padx=5)

tk.Button(
    button_frame,
    text="Cambiar Contraseña",
    command=lambda: self.cambiar_password_dialog(tree, dialog),
    bg="#153448",
    fg="white",
    font=("Arial", 10, "bold")
).pack(side=tk.LEFT, padx=5)

tk.Button(
    button_frame,
    text="Cerrar",
    command=dialog.destroy,
    bg="#443627",
    fg="white",
    font=("Arial", 10, "bold")
).pack(side=tk.RIGHT, padx=5)

```



10. Método `agregar_usuario_dialog()`:

Función: Muestra un diálogo modal para agregar nuevos usuarios al sistema. Este diálogo encapsula toda la lógica necesaria para el registro de nuevos usuarios de forma segura y user-friendly.

- Permite registrar nuevos usuarios con sus credenciales y rol (admin o no).
- Valida los datos antes de realizar el registro.

```

def agregar_usuario_dialog(self, parent):
    # Muestra diálogo para agregar nuevo usuario
    dialog = tk.Toplevel(parent)
    dialog.title("Agregar Nuevo Usuario")
    dialog.geometry("300x300")

    # Variables Tkinter para almacenar los datos del formulario
    username_var = tk.StringVar()
    password_var = tk.StringVar()
    confirm_var = tk.StringVar()
    is_admin_var = tk.BooleanVar(value=True)

    # Campos del formulario

    tk.Label(dialog, text="Nombre de usuario:").pack(pady=5)
    tk.Entry(dialog, textvariable=username_var).pack(pady=5)

    tk.Label(dialog, text="Contraseña:").pack(pady=5)
    tk.Entry(dialog, textvariable=password_var, show="*").pack(pady=5)

    tk.Label(dialog, text="Confirmar contraseña:").pack(pady=5)
    tk.Entry(dialog, textvariable=confirm_var, show="*").pack(pady=5)

    # Checkbox para definir rol de administrador
    tk.Checkbutton(
        dialog,
        text="Es administrador",
        variable=is_admin_var # Vinculado a la variable booleana
    ).pack(pady=5)

    # Función de confirmación/validación
    def confirmar():
        """Valida los datos y registra el nuevo usuario"""
        # Obtiene y limpia los datos del formulario
        username = username_var.get().strip()
        password = password_var.get()
        confirm = confirm_var.get()

        # Validación básica de campos requeridos
        if not username or not password:
            messagebox.showerror("Error", "Usuario y contraseña son requeridos")
            return

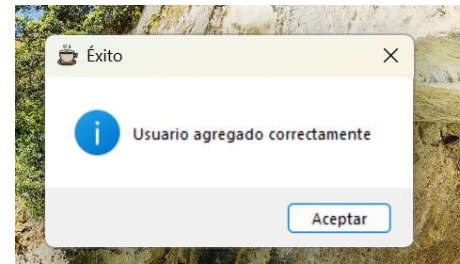
        # Validación de coincidencia de contraseñas
        if password != confirm:
            messagebox.showerror("Error", "Las contraseñas no coinciden")
            return

        try:
            # Intenta agregar el usuario mediante el UserManager
            self.user_manager.add_user(username, password, is_admin_var.get())
            messagebox.showinfo("Éxito", "Usuario agregado correctamente")
            # Cierra los diálogos y refresca la lista de usuarios
            dialog.destroy()
            parent.destroy()
            self.mostrar_gestion_usuarios()
        except ValueError as e:
            # Manejo de errores específicos (como usuario ya existente)
            messagebox.showerror("Error", str(e))

    # Botón de acción para confirmar el registro
    tk.Button(
        dialog,
        text="Agregar",
        command=confirmar,
        bg="#DBB5B5",
        fg="black"
    ).pack(pady=10)

```

Al presionar el botón de agregar:



Esto pasa en la base de datos:

```
DESCARGAS Y COSAS > users.json > ...
[
  {
    "id": "a31752bb-2bd4-49fa-b7f2-a96e5648a649",
    "username": "admin",
    "password_hash": "5640f504804c5c11fec8d030ae83cb40a7854f7904f0121aaa9489be70fd3c90",
    "is_owner": true,
    "created_at": "2025-04-14 21:02:28"
  },
  {
    "id": "d8883024-0f9e-4672-9892-50d9d8529939",
    "username": "Pazuzu",
    "password_hash": "217dd19dd292325bbbe231456ec670a892d5744b87c7b27982d32c8f2d8bfab3",
    "is_admin": true,
    "created_at": "2025-04-24 23:22:01"
  }
]
```

11. Método `agregar_usuario_dialog()`:

Función: Elimina un usuario del sistema después de confirmar la selección y gestiona la eliminación a través del UserManager y maneja posibles errores.

```
def eliminar_usuario(self, tree, parent):
    # Verificación de selección
    seleccion = tree.selection()

    # Si no hay selección, muestra advertencia y termina ejecución
    if not seleccion:
        messagebox.showwarning("Advertencia", "Por favor seleccione un usuario")
        return

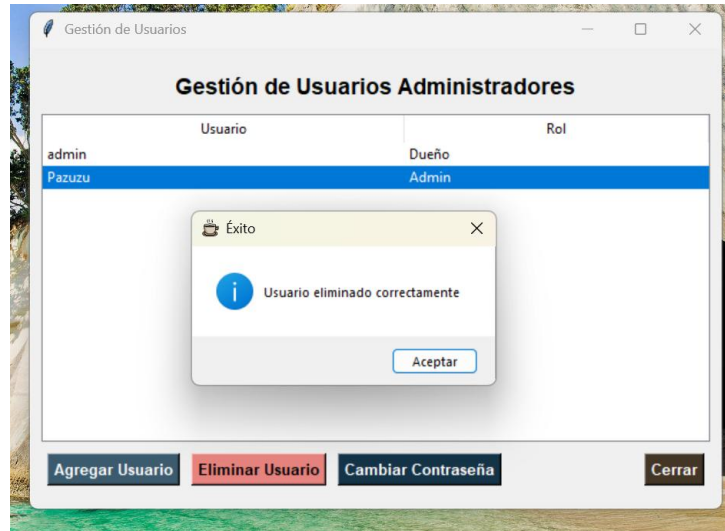
    # Obtención del nombre de usuario a eliminar
    item = seleccion[0]
    username = tree.item(item, "values")[0]

    try:
        # Intento de eliminación a través del UserManager
        self.user_manager.remove_user(username)

        # Feedback de éxito
        messagebox.showinfo("Éxito", "Usuario eliminado correctamente")

        # Cierre y actualización de la interfaz
        parent.destroy()
        self.mostrar_gestion_usuarios()

    # Manejo de excepciones específicas:
    # - PermissionError: Cuando no tiene privilegios para eliminar
    # - ValueError: Cuando intenta eliminarse a sí mismo
    except (PermissionError, ValueError) as e:
        messagebox.showerror("Error", str(e))
```



Esto pasa en la base de datos:

```
DESCARGAS Y COSAS > users.json > ...
[
  {
    "id": "a31752bb-2bd4-49fa-b7f2-a96e5648a649",
    "username": "admin",
    "password_hash": "5640f504804c5c11fec8d030ae83cb40a7854f7904f0121aaa9489be70fd3c90",
    "is_owner": true,
    "created_at": "2025-04-14 21:02:28"
  }
]
```

12. Método `cambiar_password_dialog ()`:

Función: sirve para gestionar el cambio de contraseñas de usuarios en el sistema de la cafetería, con controles de seguridad y validaciones.

Propósito Principal:

Permitir que:

- **Administradores (dueños)** cambien contraseñas de cualquier usuario.
- **Usuarios normales** cambien **solo su propia contraseña**.

```

def cambiar_password_dialog(self, tree, parent):
    # Verificación de selección de usuario
    seleccion = tree.selection()
    if not seleccion:
        messagebox.showwarning("Advertencia", "Por favor seleccione un usuario")
        return

    # Obtención del nombre de usuario seleccionado
    item = seleccion[0]
    username = tree.item(item, "values")[0]

    # Validación de permisos:
    current_user = self.user_manager.current_user["username"]
    if not self.user_manager.is_owner() and username != current_user:
        messagebox.showerror("Error", "Solo puedes cambiar tu propia contraseña")
        return

    # Configuración del diálogo de cambio de contraseña
    dialog = tk.Toplevel(parent)
    dialog.title("Cambiar Contraseña")
    dialog.geometry("300x200")

    # Variables para almacenar los datos del formulario
    new_pass_var = tk.StringVar()
    confirm_var = tk.StringVar()

    # Interfaz del formulario

    tk.Label(dialog, text=f"Cambiar contraseña para {username}").pack(pady=5)

    tk.Label(dialog, text="Nueva contraseña:").pack(pady=5)
    tk.Entry(dialog, textvariable=new_pass_var, show="*").pack(pady=5)

    tk.Label(dialog, text="Confirmar contraseña:").pack(pady=5)
    tk.Entry(dialog, textvariable=confirm_var, show="*").pack(pady=5)

    # Función de confirmación/validación
    def confirmar():
        new_pass = new_pass_var.get()
        confirm = confirm_var.get()

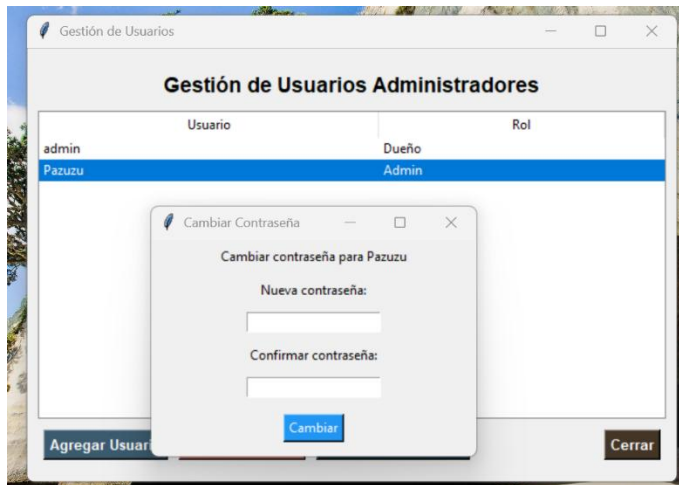
        if not new_pass or not confirm:
            messagebox.showerror("Error", "Todos los campos son requeridos")
            return

        if new_pass != confirm:
            messagebox.showerror("Error", "Las contraseñas no coinciden")
            return

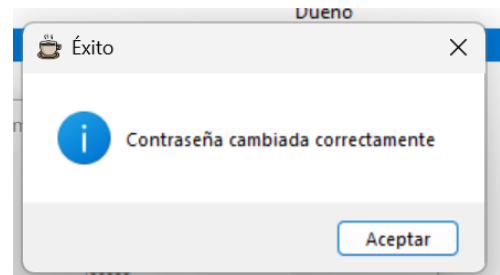
        try:
            # Intenta cambiar la contraseña a través del UserManager
            self.user_manager.change_password(username, new_pass)
            messagebox.showinfo("Éxito", "Contraseña cambiada correctamente")
            dialog.destroy()
        except Exception as e:
            messagebox.showerror("Error", str(e))

    # Botón de acción para confirmar el cambio
    tk.Button(
        dialog,
        text="Cambiar",
        command=confirmar,
        bg="#2196F3",
        fg="white"
    ).pack(pady=10)

```



Al presionar el botón de cambiar:



13. Método `cargar_pedidos()`:

Función: Cargar el historial de pedidos guardados desde un archivo JSON (pedidos.json) en la ruta especificada, para que el programa pueda acceder a estos datos durante su ejecución.

```
def cargar_pedidos(self):
    ruta_pedidos = os.path.join("D:", "DESCARGAS Y COSAS", "pedidos.json")
    try:
        with open(ruta_pedidos, 'r') as f:
            self.pedidos = json.load(f)
    except FileNotFoundError:
        self.pedidos = []
```

14. Método `guardar_pedidos()`:

Función: Guardar la lista actual de pedidos (self.pedidos) en un archivo JSON (pedidos.json) para persistir los datos entre sesiones de la aplicación.

```
def guardar_pedidos(self):
    ruta_pedidos = os.path.join("D:", "DESCARGAS Y COSAS", "pedidos.json")
    with open(ruta_pedidos, 'w') as f:
        json.dump(self.pedidos, f, indent=4)
```

Abre el archivo en modo escritura ('w'), lo que crea el archivo si no existe o lo sobrescribe si ya existe.

Ejemplo de Archivo Generado (pedidos.json):

```
DESCARGAS Y COSAS > pedidos.json > ...
[
  {
    "cliente": "Fatima",
    "fecha": "2025-04-25 00:23:53",
    "productos": [
      "Croissant"
    ],
    "cantidades": [
      1
    ],
    "total": 10.0,
    "detalle": [
      "Croissant ($10.00)"
    ]
  }
]
```

15. Método cargar_productos():

Funciones:

- Cargar productos existentes desde un archivo JSON (productos.json) y convertirlos en objetos Producto
- Inicializar con productos predeterminados si el archivo no existe
- Organizar los productos en dos categorías:
 - self.bebidas: Lista de productos de tipo bebida
 - self.postres: Lista de productos de tipo postre

```
def cargar_productos(self):
    # Define ruta del archivo JSON de productos
    ruta_productos = os.path.join("D:", "DESCARGAS Y COSAS", "productos.json")
    try:
        with open(ruta_productos, 'r') as f:
            data = json.load(f)
            self.bebidas = [Producto(**p) for p in data.get('bebidas', [])]
            self.postres = [Producto(**p) for p in data.get('postres', [])]
    except FileNotFoundError:
        # Bebidas predeterminadas
        self.bebidas = [
            Producto("Espresso", 35.00, "Bebida", os.path.join("D:", "DESCARGAS Y COSAS", "espresso.png"), {"café": 10, "agua": 30}),
            Producto("Macchiato", 40.00, "Bebida", os.path.join("D:", "DESCARGAS Y COSAS", "macchiato.png"), {"café": 10, "leche": 100}),
            Producto("Chocolate", 35.00, "Bebida", os.path.join("D:", "DESCARGAS Y COSAS", "chocolate.png"), {"chocolate": 20, "leche": 100})
        ]
        # Postres predeterminados
        self.postres = [
            Producto("Cupcake Celestial", 6.00, "Postre", os.path.join("D:", "DESCARGAS Y COSAS", "cupcake_celestial.png"), {"harina": 50, "azúcar": 10}),
            Producto("Pastel de Chocolate", 8.00, "Postre", os.path.join("D:", "DESCARGAS Y COSAS", "pastel_chocolate.png"), {"harina": 50, "chocolate": 10})
        ]
        # Guardar los productos predeterminados en el archivo JSON
        self.guardar_productos()
```

Estructura del JSON generado:

```
{
  "bebidas": [
    {
      "nombre": "Expreso",
      "precio": 35.00,
      "categoria": "Bebida",
      "imagen_path": "ruta/imagen.png",
      "ingredientes": {"café": 10, "agua": 30}
    }
  ],
  "postres": [
    {
      "nombre": "Cupcake Celestial",
      "precio": 6.00,
      "categoria": "Postre",
      "imagen_path": "ruta/imagen.png",
      "ingredientes": {"harina": 50, "huevos": 1}
    }
  ]
}
```

16. Método guardar_productos():

Función: Guarda la lista de productos (bebidas y postres) en un archivo JSON para persistir los datos entre ejecuciones del programa.

```
def guardar_productos(self):
    # Define la ruta del archivo donde se guardarán los productos
    # En este caso, se guarda en la carpeta "DESCARGAS Y COSAS" del disco D:
    ruta_productos = os.path.join("D:", "DESCARGAS Y COSAS", "productos.json")

    # Prepara los datos a guardar en un formato estructurado (diccionario)
    data = {
        # Convierte cada bebida en un diccionario con sus atributos
        'bebidas': [{ 'nombre': p.nombre, 'precio': p.precio, 'categoria': p.categoria,
                       'imagen_path': p.imagen_path, 'ingredientes': p.ingredientes }
                     for p in self.bebidas],
        # Convierte cada postre en un diccionario con sus atributos
        'postres': [{ 'nombre': p.nombre, 'precio': p.precio, 'categoria': p.categoria,
                      'imagen_path': p.imagen_path, 'ingredientes': p.ingredientes }
                     for p in self.postres]
    }

    # Abre el archivo en modo escritura ('w')
    with open(ruta_productos, 'w') as f:
        # Escribe los datos en formato JSON con indentación (4 espacios) para mejor legibilidad
        json.dump(data, f, indent=4)
```

Este método se llama cada vez que se modifica la lista de productos (bebidas o postres), como cuando:

- Se agrega un nuevo producto
- Se elimina un product
- Se editan los detalles de un producto existente

Estructura de datos:

- Organiza los productos en dos categorías: bebidas y postres
- Para cada categoría, convierte los objetos Producto en diccionarios con:
 - Nombre del product
 - Precio
 - Categoría
 - Ruta de la imagen
 - Lista de ingredientes necesarios

El archivo JSON resultante tendrá una estructura similar a esta:

```
DESCARGAS Y COSAS > productos.json > ...
{
  "bebidas": [],
  "postres": [
    {
      "nombre": "Croissant",
      "precio": 10.0,
      "categoria": "Postre",
      "imagen_path": "D:DESCARGAS Y COSAS\\croissant.png",
      "ingredientes": {
        "harina": 15.0,
        "az\u00facar": 10.0,
        "huevos": 1.0
      }
    }
  ]
}
```

17. Método realizar_pedido():

Función: Permite a los clientes seleccionar productos (bebidas y postres) y realizar un pedido. Este método es central para la funcionalidad de ventas del programa, proporcionando una experiencia de usuario completa para realizar pedidos.

```

def realizar_pedido(self):
    # Crea una nueva ventana emergente para realizar pedidos
    self.pedido_window = tk.Toplevel(self.root)
    self.pedido_window.title("Realizar Pedido")
    self.pedido_window.geometry("1000x500")
    self.pedido_window.configure(bg="#f5f5f5")

    # Inicializa variables para almacenar los productos seleccionados y el total
    self.productos_seleccionados = []
    self.total_pedido = 0.0

    # Frame principal que contiene toda la interfaz
    main_frame = tk.Frame(self.pedido_window, bg="#f5f5f5")
    main_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

    # Frame izquierdo para mostrar los productos disponibles
    left_frame = tk.Frame(main_frame, bg="#f5f5f5")
    left_frame.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

    # Notebook (pestañas) para organizar bebidas y postres
    notebook = ttk.Notebook(left_frame)
    bebidas_frame = ttk.Frame(notebook)
    notebook.add(bebidas_frame, text="Bebidas")
    postres_frame = ttk.Frame(notebook)
    notebook.add(postres_frame, text="Postres")
    notebook.pack(fill=tk.BOTH, expand=True)

    # Función interna para configurar el frame de productos
    def setup_product_frame(parent, productos):
        # Configuración de scroll para manejar muchos productos
        canvas = tk.Canvas(parent, bg="#f5f5f5", highlightthickness=0)
        scrollbar = tk.Scrollbar(parent, orient="vertical", command=canvas.yview)
        scrollable_frame = tk.Frame(canvas, bg="#f5f5f5")

        # Configuración del sistema de scroll
        scrollable_frame.bind("<Configure>", lambda e: canvas.configure(scrollregion=canvas.bbox("all")))
        canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
        canvas.configure(yscrollcommand=scrollbar.set)

        scrollbar.pack(side="right", fill="y")
        canvas.pack(side="left", fill="both", expand=True)

        # Configuración de columnas para organizar los productos en una cuadrícula
        for i in range(3):
            scrollable_frame.grid_columnconfigure(i, weight=1, uniform="cols")

        # Mostrar cada producto en un frame individual
        for i, producto in enumerate(productos):
            frame = tk.Frame(scrollable_frame, bg="white", bd=1, relief="solid", padx=10, pady=10, width=150, height=220)
            frame.grid(row=i//3, column=i%3, padx=10, pady=10, sticky="nsew")
            frame.grid_propagate(False)

            try:
                # Cargar y mostrar la imagen del producto
                img_path = producto.imagen_path
                if not os.path.exists(img_path):
                    img_path = os.path.join("images", "default.png")
                    if not os.path.exists(img_path):
                        raise FileNotFoundError("Imagen no encontrada")

                imagen = Image.open(img_path).resize((120, 120), Image.Resampling.LANCZOS)
                imagen_tk = ImageTk.PhotoImage(imagen)
                self.imagenes_tk.append(imagen_tk)

                # Mostrar elementos del producto
                tk.Label(frame, image=imagen_tk, bg="white").pack(pady=5)
                tk.Label(frame, text=producto.nombre, bg="white", font=("Arial", 10, "bold"), wraplength=120).pack()
                tk.Label(frame, text=f"${producto.precio:.2f}", bg="white", font=("Arial", 10, "bold"), fg="#443627").pack(pady=5)

                # Botón para agregar producto al pedido
                tk.Button(
                    frame,
                    text="Agregar",
                    command=lambda p=producto: self.agregar_producto_pedido(p),
                    bg="#443627", fg="white", relief="flat", font=("Arial", 9), padx=15, pady=3
                ).pack(pady=5, fill=tk.X)
            except:
                pass

```

```

def setup_product_frame(parent, productos):
    except Exception as e:
        print(f"Error al cargar la imagen: {e}")
        tk.Label(frame, text="Imagen no disponible", bg="white").pack()

    # Configurar frames para bebidas y postres
    setup_product_frame(bebidas_frame, self.bebidas)
    setup_product_frame(postres_frame, self.postres)

    # Frame derecho para mostrar los productos seleccionados y el total
    right_frame = tk.Frame(main_frame, bg="white", bd=1, relief="solid", width=300)
    right_frame.pack(side=tk.RIGHT, fill=tk.Y, padx=(20, 0))

    # Título de la sección de productos seleccionados
    tk.Label(right_frame, text="Productos seleccionados", bg="white", font=("Arial", 12, "bold"), pady=10).pack(fill=tk.X)

    # Frame con scroll para la lista de productos seleccionados
    selected_list_frame = tk.Frame(right_frame, bg="white")
    selected_list_frame.pack(fill=tk.BOTH, expand=True, padx=10)

    selected_canvas = tk.Canvas(selected_list_frame, bg="white", highlightthickness=0)
    selected_scroll = tk.Scrollbar(selected_list_frame, orient="vertical", command=selected_canvas.yview)
    self.selected_items_frame = tk.Frame(selected_canvas, bg="white")

    self.selected_items_frame.bind("<Configure>", lambda e: selected_canvas.configure(scrollregion=selected_canvas.bbox("all")))
    selected_canvas.create_window((0, 0), window=self.selected_items_frame, anchor="nw")
    selected_canvas.configure(yscrollcommand=selected_scroll.set)

    selected_scroll.pack(side="right", fill="y")
    selected_canvas.pack(side="left", fill="both", expand=True)

    # Etiqueta para mostrar el total del pedido
    self.total_label = tk.Label(right_frame, text="Total: $0.00", bg="white", font=("Arial", 12, "bold"), pady=10)
    self.total_label.pack(fill=tk.X)

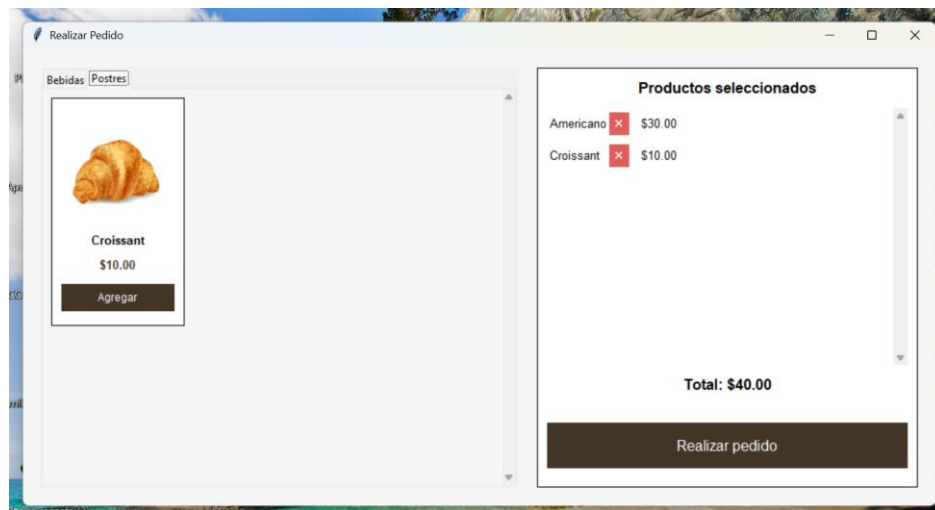
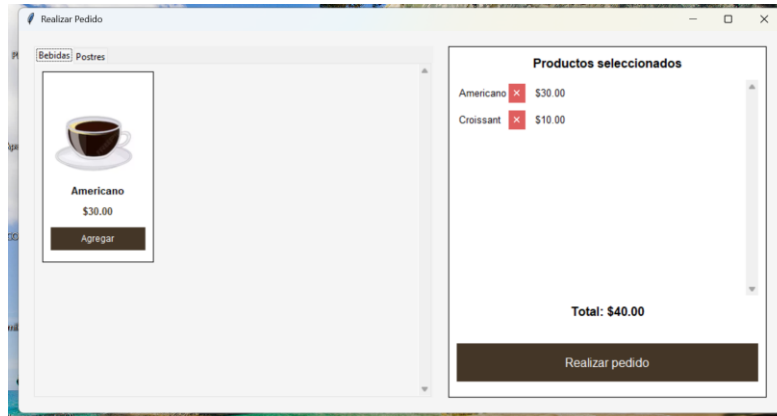
    # Botón para finalizar el pedido
    tk.Button(
        right_frame,
        text="Realizar pedido",
        command=lambda: self.finalizar_pedido(),
        bg="#443627", fg="white", font=("Arial", 12), relief="flat", padx=20, pady=10
    ).pack(fill=tk.X, padx=10, pady=20)

    # Actualizar el resumen del pedido (inicialmente vacío)
    self.actualizar_resumen()

```

Propósito principal: Este método crea una interfaz gráfica que permite:

1. **Explorar productos disponibles** organizados en pestañas (bebidas y postres)
2. **Seleccionar productos** para agregarlos al pedido



3. **Ver el resumen del pedido** con los productos seleccionados y el total a pagar
4. **Confirmar el pedido** cuando esté listo

18. Método `agregar_producto_pedido()`:

Propósito:

Este método es fundamental para la gestión de pedidos ya que:

- Gestiona la disponibilidad de productos: Verifica que existan suficientes ingredientes en el inventario para preparar el producto solicitado.
- Actualiza el inventario: Si el producto está disponible, descuenta los ingredientes usados del inventario.
- Actualiza el pedido: Agrega el producto a la lista de seleccionados y calcula el nuevo total.

- Proporciona retroalimentación: Informa al usuario si algún producto no está disponible.

```
def agregar_producto_pedido(self, producto):
    try:
        # Primero se verifica si hay suficiente inventario para la preparacion del producto
        for ingrediente, cantidad in producto.ingredientes.items():
            if not self.inventario.verificar_disponibilidad(ingrediente, cantidad):
                # Si no hay suficiente, lanza una excepción con un mensaje descriptivo
                raise ValueError(f"No hay suficiente {ingrediente} para preparar {producto.nombre}")

        # Si pasa la verificación, consumimos los ingredientes y agregamos al pedido
        self.inventario.consumir_ingredientes(producto.ingredientes)
        self.productos_seleccionados.append(producto)
        self.total_pedido += producto.precio
        self.actualizar_resumen()
    except ValueError as e:
        # Se muestra el error pero se permite continuar
        messagebox.showerror("Producto no disponible", str(e))
```

19. Método actualizar_resumen():

Propósito:

Este método es responsable de:

- Actualizar visualmente la lista de productos seleccionados en el pedido
- Mostrar información clave de cada producto (nombre y precio)
- Proporcionar control para eliminar productos no deseados
- Calcular y mostrar el total acumulado del pedido

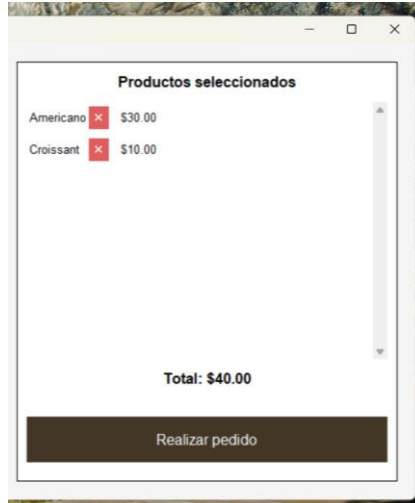
```
def actualizar_resumen(self):
    # Elimina todos los widgets existentes en el frame de productos seleccionados
    for widget in self.selected_items_frame.winfo_children():
        widget.destroy()

    # MUESTRA PRODUCTOS SELECCIONADOS
    for i, producto in enumerate(self.productos_seleccionados):
        item_frame = tk.Frame(self.selected_items_frame, bg="white", pady=5)
        item_frame.pack(fill=tk.X)

        # Muestra nombre del producto (alineado a la izquierda)
        tk.Label(item_frame, text=producto.nombre, bg="white", anchor="w", font=("Arial", 10)).pack(side=tk.LEFT, fill=tk.X, expand=True)
        # Muestra precio del producto (alineado a la derecha)
        tk.Label(item_frame, text=f"${producto.precio:.2f}", bg="white", font=("Arial", 10)).pack(side=tk.RIGHT, padx=10)

        # Botón para eliminar producto
        tk.Button(
            item_frame,
            text="X",
            command=lambda idx=i: self.eliminar_producto(idx),
            bg="#DF5E5E", fg="white", font=("Arial", 8), relief="flat", width=2
        ).pack(side=tk.RIGHT)

    # ACTUALIZAR TOTAL DEL PEDIDO
    self.total_label.config(text=f"Total: ${self.total_pedido:.2f}")
```



Relación con otros métodos

Se llama automáticamente cuando:

- Se agrega un producto (`agregar_producto_pedido()`)
- Se elimina un producto (`eliminar_producto()`)
- Hay cambios en el pedido

Actualiza elementos visuales que incluyen:

- `self.selected_items_frame`: Frame contenedor de los productos
- `self.total_label`: Etiqueta que muestra el total

20. Método `eliminar_producto()`:

Propósito:

Este método cumple varias funciones importantes:

- Gestión del pedido: Elimina un producto específico de la lista de productos seleccionados
- Actualización financiera: Recalcula el total a pagar restando el precio del producto eliminado
- Gestión de inventario: Devuelve al stock los ingredientes que se habrían usado para ese producto
- Actualización de UI: Refresca la interfaz para mostrar los cambios realizados

```
def eliminar_producto(self, index):
    # ELIMINAR PRODUCTO DE LA LISTA
    producto = self.productos_seleccionados.pop(index)
    # ACTUALIZAR TOTAL DEL PEDIDO
    self.total_pedido -= producto.precio

    # REVERTIR INVENTARIO
    for ingrediente, cantidad in producto.ingredientes.items():
        self.inventario.agregar_ingredientes(ingrediente, cantidad)
    # ACTUALIZAR INTERFAZ
    self.actualizar_resumen()
```

21. Método finalizar_pedido():

Propósito:

Este método es el proceso final del flujo de pedidos que:

- Valida que el pedido tenga productos
- Crea un registro estructurado del pedido
- Guarda la información permanentemente
- Informa al cliente con detalles del pedido
- Maneja errores y revierte cambios si es necesario
- Realiza limpieza de la interfaz

```
def finalizar_pedido(self):
    # Verificar que hay productos seleccionados
    if not self.productos_seleccionados:
        messagebox.showwarning("Pedido vacío", "No has seleccionado ningún producto para tu pedido.")
        return

    try:
        # Crear registro del pedido
        resumen = {
            "cliente": self.nombre_cliente,
            "fecha": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            "productos": [p.nombre for p in self.productos_seleccionados],
            "cantidades": [1 for _ in self.productos_seleccionados], # Puedes modificar esto si manejas cantidades
            "total": round(self.total_pedido, 2),
            "detalle": [f"{p.nombre} (${p.precio:.2f})" for p in self.productos_seleccionados]
        }

        # Agregar a la Lista de pedidos
        self.pedidos.append(resumen)

        # Intentar guardar los pedidos
        try:
            self.guardar_pedidos()
        except Exception as e:
            # Si falla el guardado, revertir los cambios en el inventario
            for producto in self.productos_seleccionados:
                for ingrediente, cantidad in producto.ingredientes.items():
                    self.inventario.agregar_ingredientes(ingrediente, cantidad)
            messagebox.showerror("Error", f"No se pudo guardar el pedido: {str(e)}")
            return

        # Mostrar confirmación detallada
        detalles_pedido = "\n".join([f"- {p.nombre}: ${p.precio:.2f}" for p in self.productos_seleccionados])

        respuesta = messagebox.askyesno(
            "Pedido Completado",
            f"¡Pedido registrado con éxito!\n\n"
            f"Cliente: {self.nombre_cliente}\n\n"
            f"Productos:\n{detalles_pedido}\n\n"
            f"Total: ${self.total_pedido:.2f}\n\n"
            f"¿Desea imprimir el ticket?",
            parent=self.pedido_window
        )
    )
```

```

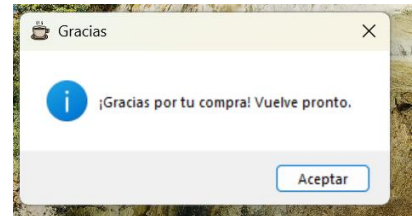
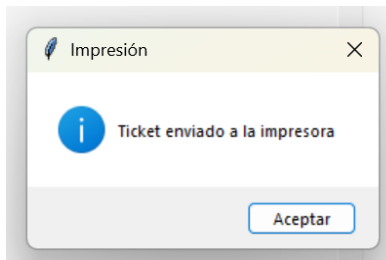
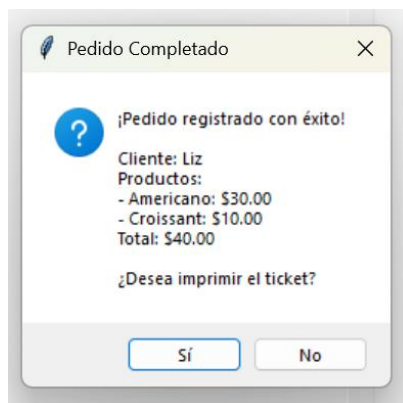
if respuesta:
    # Aquí puedes agregar la lógica para imprimir el ticket
    messagebox.showinfo("Impresión", "Ticket enviado a la impresora", parent=self.pedido_window)

    # Cerrar ventana de pedido
    self.pedido_window.destroy()

    # Opcional: Mostrar mensaje de agradecimiento
    messagebox.showinfo("Gracias", "¡Gracias por tu compra! Vuelve pronto.", parent=self.root)

except Exception as e:
    # Manejo de cualquier otro error inesperado
    messagebox.showerror("Error", f"Ocurrió un error inesperado: {str(e)}", parent=self.pedido_window)
    # Intentar revertir cambios en inventario
    try:
        for producto in self.productos_seleccionados:
            for ingrediente, cantidad in producto.ingredientes.items():
                self.inventario.agregar_ingrediente(ingrediente, cantidad)
    except Exception as ex:
        messagebox.showerror("Error Crítico", f"No se pudieron revertir los cambios: {str(ex)}", parent=self.pedido_window)

```



Relación con otros componentes

- **Inventario:** Mantiene consistencia al revertir cambios si es necesario
- **Historial de ventas:** El pedido queda registrado para reportes futuros
- **Interfaz:** Proporciona retroalimentación visual en cada paso
- **Persistencia:** Usa `guardar_pedidos()` para almacenamiento permanente

22. Método `agregar_producto()`:

Propósito:

Este método crea una interfaz completa para agregar nuevos productos al sistema de la cafetería, permitiendo:

- Definir propiedades básicas del producto (nombre, precio, categoría, imagen)
- Seleccionar ingredientes necesarios y sus cantidades
- Gestionar la lista de ingredientes seleccionados
- Confirmar la creación del nuevo producto

```
def agregar_producto(self):
    # Oculta la ventana principal (administrador)
    self.root.withdraw()

    dialog = tk.Toplevel(self.root)
    dialog.title("Agregar Producto")
    dialog.geometry("500x600")

    # Configurar qué hacer al cerrar el diálogo
    def on_close():
        dialog.destroy()
        self.root.deiconify() # <--- Restaura la ventana principal

    dialog.protocol("WM_DELETE_WINDOW", on_close) # <--- Maneja el cierre

    # Variables para el nuevo producto
    nombre_var = tk.StringVar()
    precio_var = tk.StringVar()
    categoria_var = tk.StringVar(value="Bebida")
    imagen_var = tk.StringVar()

    # Diccionario para almacenar los ingredientes seleccionados
    self.ingredientes_seleccionados = {}

    # Frame principal con scroll
    main_frame = tk.Frame(dialog)
    main_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    canvas = tk.Canvas(main_frame)
    scrollbar = ttk.Scrollbar(main_frame, orient="vertical", command=canvas.yview)
    scrollable_frame = tk.Frame(canvas)

    scrollable_frame.bind("<Configure>", lambda e: canvas.configure(scrollregion=canvas.bbox("all")))
    canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
    canvas.configure(yscrollcommand=scrollbar.set)

    # Información básica del producto
    tk.Label(scrollable_frame, text="Nombre:").grid(row=0, column=0, padx=5, pady=5, sticky="e")
    tk.Entry(scrollable_frame, textvariable=nombre_var).grid(row=0, column=1, padx=5, pady=5, sticky="w")

    tk.Label(scrollable_frame, text="Precio:").grid(row=1, column=0, padx=5, pady=5, sticky="e")
    tk.Entry(scrollable_frame, textvariable=precio_var).grid(row=1, column=1, padx=5, pady=5, sticky="w")

    tk.Label(scrollable_frame, text="Categoría:").grid(row=2, column=0, padx=5, pady=5, sticky="e")
    tk.OptionMenu(scrollable_frame, categoria_var, "Bebida", "Postre").grid(row=2, column=1, padx=5, pady=5, sticky="w")

    tk.Label(scrollable_frame, text="Imagen (nombre.extension):").grid(row=3, column=0, padx=5, pady=5, sticky="e")
    tk.Entry(scrollable_frame, textvariable=imagen_var).grid(row=3, column=1, padx=5, pady=5, sticky="w")

    # Sección de ingredientes
    tk.Label(scrollable_frame, text="Ingredientes:", font=("Arial", 10, "bold")).grid(row=4, column=0, colspan=2, pady=10)
```

```

# Frame para la lista de ingredientes disponibles
ingredientes_frame = tk.Frame(scrollable_frame)
ingredientes_frame.grid(row=5, column=0, columnspan=2, sticky="ew")

# Encabezados de la tabla de ingredientes
tk.Label(ingredientes_frame, text="Ingrediente", font=("Arial", 9, "bold")).grid(row=0, column=0, padx=5)
tk.Label(ingredientes_frame, text="Cantidad", font=("Arial", 9, "bold")).grid(row=0, column=1, padx=5)
tk.Label(ingredientes_frame, text="Unidad", font=("Arial", 9, "bold")).grid(row=0, column=2, padx=5)

# Crea un diccionario para mantener referencia a los Entry widgets
self.entry_ingredientes = {}

# Muestra todos los ingredientes disponibles
for i, (ingrediente, datos) in enumerate(self.inventario.obtener_estado().items(), 1):
    tk.Label(ingredientes_frame, text=ingrediente).grid(row=i, column=0, padx=5, pady=2, sticky="w")

    # Crea un Entry para cada ingrediente
    entry_cantidad = tk.Entry(ingredientes_frame, width=8)
    entry_cantidad.grid(row=i, column=1, padx=5, pady=2)
    self.entry_ingredientes[ingrediente] = entry_cantidad # Guardamos referencia

    tk.Label(ingredientes_frame, text=datos['unidad']).grid(row=i, column=2, padx=5, pady=2)

    # Botón para agregar cada ingrediente (usa lambda con ingrediente y entry específicos)
    tk.Button(
        ingredientes_frame,
        text="Agregar",
        command=lambda ing=ingrediente, entry=entry_cantidad: self.agregar_ingrediente_producto(ing, entry),
        bg="#DBB5B5",
        fg="black",
        font=("Arial", 8)
    ).grid(row=i, column=3, padx=5, pady=2)

# Frame para ingredientes seleccionados
seleccionados_frame = tk.Frame(scrollable_frame)
seleccionados_frame.grid(row=6, column=0, columnspan=2, pady=10, sticky="ew")

tk.Label(seleccionados_frame, text="Ingredientes seleccionados:", font=("Arial", 10, "bold")).pack(anchor="w")

self.lista_seleccionados = tk.Listbox(seleccionados_frame, height=4)
self.lista_seleccionados.pack(fill=tk.X, pady=5)

def eliminar_seleccionado():
    seleccion = self.lista_seleccionados.curselection()
    if seleccion:
        ingrediente = self.lista_seleccionados.get(seleccion[0]).split(":")[0].strip()
        del self.ingredientes_seleccionados[ingrediente]
        self.actualizar_lista_seleccionados()

tk.Button(
    seleccionados_frame,
    text="Eliminar seleccionado",
    command=eliminar_seleccionado,
    bg="#F1E5D1",
    fg="black",
    font=("Arial", 8)
).pack(pady=5)

# Botón para confirmar
tk.Button(
    scrollable_frame,
    text="Agregar Producto",
    command=lambda: [self.confirmar_agregar_producto(nombre_var, precio_var, categoria_var, imagen_var, dialog), on_close()],
    bg="#443627",
    fg="white",
    font=("Arial", 10, "bold"),
    pady=10
).grid(row=7, column=0, columnspan=2, pady=20, sticky="ew")

canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

```

Nombre:

Precio:

Categoria:

Imagen (nombre.extension):

Ingredientes:

Ingrediente	Cantidad	Unidad	
leche		ml	Agregar
harina		gr	Agregar
café		gr	Agregar
chocolate		gr	Agregar
azúcar		gr	Agregar
huevos		unidades	Agregar
agua		ml	Agregar
limon		unidades	Agregar
vainilla		ml	Agregar

Ingredientes seleccionados:

Relación con Otros Componentes

- **Inventario:** Usa los ingredientes disponibles para mostrar opciones
- **Productos:** Prepara datos para crear nuevos objetos Producto
- **Interfaz principal:** Se integra con el flujo de administración
- **Persistencia:** Conecta con métodos para guardar los nuevos productos

23. Método `agregar_ingrediente_producto()`:

Propósito:

Este método sirve para:

- **Agregar ingredientes** a la lista de componentes de un nuevo producto
- **Validar** que las cantidades ingresadas sean correctas
- **Mantener actualizado** el diccionario de ingredientes seleccionados
- **Proporcionar retroalimentación** al usuario sobre errores en los datos

```
def agregar_ingrediente_producto(self, ingrediente, entry_cantidad):
    # Obtiene el valor del campo de entrada de cantidad
    cantidad = entry_cantidad.get()
    # Verifica que se haya ingresado una cantidad (no vacío)
    if cantidad:
        try:
            # Intenta convertir la cantidad a número flotante
            cantidad_float = float(cantidad)
            # Valida que la cantidad sea mayor que cero
            if cantidad_float <= 0:
                messagebox.showerror("Error", "La cantidad debe ser mayor que cero")
                return
            # Agrega el ingrediente al diccionario de seleccionados
            self.ingredientes_seleccionados[ingrediente] = cantidad_float
            # Limpia el campo de entrada para nueva entrada
            entry_cantidad.delete(0, tk.END)
            # Actualiza la lista visual de ingredientes seleccionados
            self.actualizar_lista_seleccionados()
        except ValueError:
            # Maneja el error si la conversión a número falla
            messagebox.showerror("Error", "La cantidad debe ser un número válido")
```

Relación con Otros Componentes

- **Interfaz de agregar producto:** Proporciona la entrada de datos
- `actualizar_lista_seleccionados()`: Actualiza la visualización
- `ingredientes_seleccionados`: Diccionario que mantiene el estado actual

24. Método `actualizar_lista_seleccionados()`:

Propósito:

Este método sirve para:

- Sincronizar la lista visual de ingredientes con los datos actuales
- Mostrar de forma clara y organizada:
 1. Nombre del ingrediente
 2. Cantidad requerida
 3. Unidad de medida correspondiente

```
def actualizar_lista_seleccionados(self):
    # Limpia la lista actual eliminando todos sus elementos
    self.lista_seleccionados.delete(0, tk.END)
    # Recorre todos los ingredientes seleccionados
    for ingrediente, cantidad in self.ingredientes_seleccionados.items():
        # Obtiene la unidad de medida del ingrediente desde el inventario
        unidad = self.inventario.obtener_ingrediente(ingrediente)['unidad']
        # Agrega el ingrediente formateado a la lista visual
        self.lista_seleccionados.insert(tk.END, f"{ingrediente}: {cantidad} {unidad}")
```

Relación con Otros Componentes

- `self.lista_seleccionados`: El widget Listbox que muestra los ingredientes

- self.ingredientes_seleccionados: Diccionario con los ingredientes actuales
- self.inventario: Proporciona las unidades de medida

25. Método actualizar_lista_seleccionados():

Propósito:

Este método es el punto final del flujo de agregar productos y sirve para:

- Validar todos los datos ingresados por el usuario
- Crear el objeto Producto con la información proporcionada
- Guardar el producto en la categoría correspondiente (Bebidas/Postres)
- Persistir los cambios en el almacenamiento
- Manejar errores y proporcionar retroalimentación
- Cerrar el diálogo al completar el proceso

```
def confirmar_agregar_producto(self, nombre_var, precio_var, categoria_var, imagen_var, dialog):
    try:
        # VALIDACIÓN DE DATOS BÁSICOS
        # Obtiene y valida el nombre (no puede estar vacío)
        nombre = nombre_var.get().strip()
        if not nombre:
            raise ValueError("El nombre no puede estar vacío")

        # Obtiene y valida el precio (debe ser número positivo)
        precio = float(precio_var.get())
        if precio <= 0:
            raise ValueError("El precio debe ser mayor que cero")

        # Obtiene categoría e imagen
        categoria = categoria_var.get()
        imagen_nombre = imagen_var.get().strip()

        # Valida que haya al menos un ingrediente seleccionado
        if not self.ingredientes_seleccionados:
            raise ValueError("Debe seleccionar al menos un ingrediente")

        # CONSTRUCCIÓN DE RUTA DE IMAGEN
        imagen_path = os.path.join("D:", "DESCARGAS Y COSAS", imagen_nombre)

        # CREACIÓN DEL NUEVO PRODUCTO
        nuevo_producto = Producto(nombre, precio, categoria, imagen_path, self.ingredientes_seleccionados.copy())
        # ALMACENAMIENTO SEGÚN CATEGORÍA
        if categoria == "Bebida":
            self.bebidas.append(nuevo_producto)
        else:
            self.postres.append(nuevo_producto)

        # GUARDADO EN ARCHIVO
        self.guardar_productos()

        # CONFIRMACIÓN Y CIERRE
        messagebox.showinfo("Éxito", "Producto agregado correctamente")
        dialog.destroy()
    except ValueError as e:
        messagebox.showerror("Error", f"Datos inválidos: {str(e)}")
    except Exception as e:
```

Relación con Otros Componentes

- Interfaz: Recibe variables Tkinter del formulario
- Productos: Crea nuevas instancias de la clase Producto
- Persistencia: Usa guardar_productos() para almacenamiento

- Inventario: Considera los ingredientes seleccionados

26. Método `mostrar_inventario_productos()`:

Propósito:

Este método sirve para:

- Mostrar una interfaz dedicada para gestionar todos los productos del sistema
- Organizar el espacio para visualizar bebidas y postres
- Permitir operaciones de edición y eliminación
- Mantener un flujo coherente entre ventanas

```
def mostrar_inventario_productos(self):
    """Muestra una ventana con la lista de productos existentes para editar/eliminar"""
    # Cerrar la ventana actual de gestión
    self.root.withdraw()

    # Crear nueva ventana para el inventario
    self.inventario_window = tk.Toplevel()
    self.inventario_window.title("Inventario de Productos")
    self.inventario_window.geometry("1000x600")

    # Configurar qué hacer al cerrar esta ventana
    def on_close():
        self.inventario_window.destroy()
        self.root.deiconify()

    self.inventario_window.protocol("WM_DELETE_WINDOW", on_close)

    # Frame principal que contendrá todas las vistas
    self.main_container = tk.Frame(self.inventario_window)
    self.main_container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    # Mostrar la vista de lista inicialmente
    self.mostrar_lista_productos()
```

Relación con Otros Componentes

- Ventana principal: Se oculta temporalmente
- `mostrar_lista_productos()`: Muestra el contenido inicial
- `inventario_window`: Ventana dedicada que se crea
- `main_container`: Área flexible para diferentes vistas

27. Método `mostrar_lista_productos()`:

Propósito:

Este método sirve para:

- Organizar y mostrar todos los productos del sistema en una interfaz estructurada
- Separar visualmente los productos por categorías (bebidas/postres) usando pestañas
- Proporcionar acceso a las funciones de gestión de productos
- Permitir el cierre controlado de la ventana de inventario

```
def mostrar_lista_productos(self):
    """Muestra la lista de productos en el contenedor principal"""
    # Limpiar el contenedor principal
    for widget in self.main_container.winfo_children():
        widget.destroy()

    # Notebook para categorías
    notebook = ttk.Notebook(self.main_container)
    notebook.pack(fill=tk.BOTH, expand=True)

    # Pestaña para Bebidas
    bebidas_frame = ttk.Frame(notebook)
    notebook.add(bebidas_frame, text="Bebidas")
    self._crear_tabla_productos(bebidas_frame, self.bebidas)

    # Pestaña para Postres
    postres_frame = ttk.Frame(notebook)
    notebook.add(postres_frame, text="Postres")
    self._crear_tabla_productos(postres_frame, self.postres)

    # Botón para cerrar
    tk.Button(
        self.main_container,
        text="Cerrar",
        command=lambda: [self.inventario_window.destroy(), self.root.deiconify()],
        bg="#443627",
        fg="white",
        font=("Arial", 10, "bold"),
        pady=5
    ).pack(pady=10)
```

Relación con Otros Componentes

- `self.main_container`: Contenedor principal donde se muestra la lista
- `self._crear_tabla_productos()`: Método que genera las tablas interactivas
- `self.inventario_window`: Ventana que contiene este listado
- `self.root`: Ventana principal que se restaura al cerrar

28. Método `_crear_tabla_productos()`:

Función: crear y configurar una tabla interactiva (Treeview) para mostrar los productos de la cafetería, junto con botones para editarlos o eliminarlos.

Botones de acción:

- **Editar:** Llama a `_mostrar_formulario_edicion` con el producto seleccionado.
- **Eliminar:** Llama a `_eliminar_producto` con el producto seleccionado.
- **Doble clic:** También permite editar al hacer doble clic en una fila.

```
def _crear_tabla_productos(self, parent, productos):
    """Crea la tabla de productos con opciones de editar/eliminar"""
    # Frame para la tabla
    table_frame = tk.Frame(parent)
    table_frame.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

    # Scrollbars
    scroll_y = tk.Scrollbar(table_frame)
    scroll_y.pack(side=tk.RIGHT, fill=tk.Y)

    scroll_x = tk.Scrollbar(table_frame, orient=tk.HORIZONTAL)
    scroll_x.pack(side=tk.BOTTOM, fill=tk.X)

    # Treeview
    columns = ("Nombre", "Precio", "Ingredientes")
    tree = ttk.Treeview(
        table_frame,
        columns=columns,
        yscrollcommand=scroll_y.set,
        xscrollcommand=scroll_x.set,
        selectmode="browse"
    )

    # Configurar columnas
    tree.heading("#0", text="ID")
    tree.column("#0", width=50, stretch=tk.NO)

    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, width=200 if col == "Ingredientes" else 100)

    # Insertar datos
    for i, producto in enumerate(productos, 1):
        ingredientes = ", ".join([f"{k}:{v}" for k, v in producto.ingredientes.items()])
        tree.insert("", tk.END, text=str(i),
                    values=(producto.nombre, f"${producto.precio:.2f}", ingredientes))

    tree.pack(fill=tk.BOTH, expand=True)
    scroll_y.config(command=tree.yview)
    scroll_x.config(command=tree.xview)

    # Frame para botones de acción
    button_frame = tk.Frame(parent)
    button_frame.pack(fill=tk.X, padx=5, pady=5)

    # Botones
    tk.Button(
        button_frame,
        text="Editar Producto",
        command=lambda: self._mostrar_formulario_edicion(tree, productos),
        bg="#D8B5B5",
        fg="black",
        font=("Arial", 10, "bold")
    ).pack(side=tk.LEFT, padx=5)
```

```

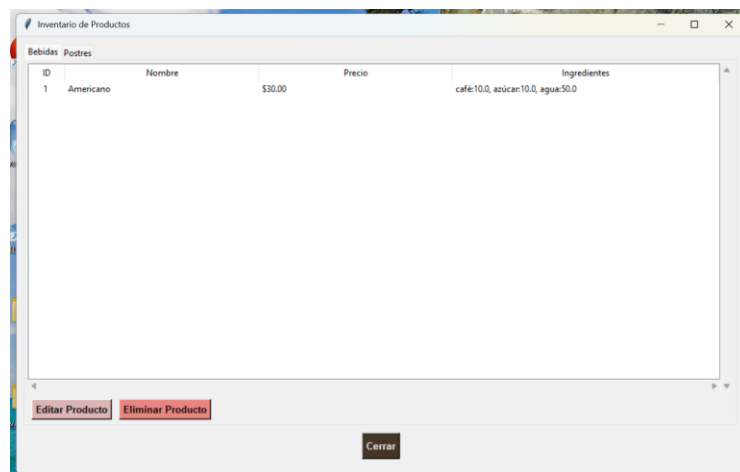
tk.Button(
    button_frame,
    text="Eliminar Producto",
    command=lambda: self._eliminar_producto(tree, productos),
    bg="#E98580",
    fg="black",
    font=("Arial", 10, "bold")
).pack(side=tk.LEFT, padx=5)

# Configurar doble clic para editar
tree.bind("<Double-1>", lambda e: self._mostrar_formulario_edicion(tree, productos))

```

Datos importantes:

- El guion bajo al inicio del nombre (_crear_tabla_productos) indica que es un método **interno** de la clase.
- La tabla creada es interactiva: los cambios se reflejan en las listas self.bebidas o self.postres y se guardan en el archivo JSON al confirmar.



29. Método _mostrar_formulario_edicion():

Función: mostrar un formulario interactivo para editar los detalles de un producto seleccionado.

Botones de acción:

- **Guardar Cambios:** Ejecuta _guardar_cambios_producto() con los nuevos datos.
- **Cancelar:** Vuelve a la lista de productos sin guardar cambios.

```

def _mostrar_formulario_edicion(self, tree, productos):
    """Muestra el formulario de edición en el mismo contenedor"""
    seleccion = tree.selection()
    if not seleccion:
        messagebox.showwarning("Advertencia", "Por favor seleccione un producto para editar")
        return

    item = seleccion[0]
    idx = int(tree.index(item))
    self.producto_actual = productos[idx]

    # Limpiar el contenedor principal
    for widget in self.main_container.winfo_children():
        widget.destroy()

    # Frame principal del formulario
    form_frame = tk.Frame(self.main_container)
    form_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    # Título
    tk.Label(
        form_frame,
        text=f"Editando: {self.producto_actual.nombre}",
        font=("Arial", 14, "bold")
    ).pack(pady=10)

    # Campos del formulario
    campos_frame = tk.Frame(form_frame)
    campos_frame.pack(fill=tk.X, pady=10)

    # Nombre
    tk.Label(campos_frame, text="Nombre:").grid(row=0, column=0, sticky="e", padx=5, pady=5)
    nombre_entry = tk.Entry(campos_frame)
    nombre_entry.grid(row=0, column=1, sticky="ew", padx=5, pady=5)
    nombre_entry.insert(0, self.producto_actual.nombre)

    # Precio
    tk.Label(campos_frame, text="Precio:").grid(row=1, column=0, sticky="e", padx=5, pady=5)
    precio_entry = tk.Entry(campos_frame)
    precio_entry.grid(row=1, column=1, sticky="ew", padx=5, pady=5)
    precio_entry.insert(0, str(self.producto_actual.precio))

    # Categoría
    tk.Label(campos_frame, text="Categoría:").grid(row=2, column=0, sticky="e", padx=5, pady=5)
    categoria_var = tk.StringVar(value=self.producto_actual.categoria)
    tk.OptionMenu(campos_frame, categoria_var, "Bebida", "Postre").grid(row=2, column=1, sticky="w", padx=5, pady=5)

    # Frame para ingredientes
    ingredientes_frame = tk.LabelFrame(form_frame, text="Ingredientes", padx=5, pady=5)
    ingredientes_frame.pack(fill=tk.X, pady=10)

    # Lista de ingredientes
    self.ingredientes_entries = {}
    for i, (ingrediente, cantidad) in enumerate(self.producto_actual.ingredientes.items()):
        tk.Label(ingredientes_frame, text=f"{ingrediente}:").grid(row=i, column=0, sticky="e", padx=5)
        entry = tk.Entry(ingredientes_frame, width=10)
        entry.grid(row=i, column=1, sticky="w", padx=5)
        entry.insert(0, str(cantidad))
        tk.Label(ingredientes_frame, text=self.inventario.obtener_ingrediente(ingrediente)['unidad']).grid(row=i, column=2, sticky="e", padx=5)
        self.ingredientes_entries[ingrediente] = entry

    # Botones
    button_frame = tk.Frame(form_frame)
    button_frame.pack(pady=10)

    tk.Button(
        button_frame,
        text="Guardar Cambios",
        command=lambda: self._guardar_cambios_producto(
            nombre_entry.get(),
            precio_entry.get(),
            categoria_var.get()
        ),
        bg="#D8B5B5",
        fg="Black"
    ).pack(side=tk.LEFT, padx=5)

    tk.Button(
        button_frame,
        text="Cancelar",
        command=self.mostrar_lista_productos,
        bg="#D8B5B5",
        fg="Black"
    ).pack(side=tk.LEFT, padx=5)

```

Datos importantes:

- Dependencias:
 - Requiere que `self.main_container` exista (normalmente es un Frame de la ventana de inventario).
 - Usa `self.inventario` para obtener las unidades de medida de los ingredientes.

The screenshot shows a window titled 'Inventario de Productos'. Inside, there's a section titled 'Editando: Americano'. It contains three input fields: 'Nombre' with the value 'Americano', 'Precio' with the value '30.0', and 'Categoría' with a dropdown menu showing 'Bebida'. Below these is a section titled 'Ingredientes' which contains three rows of input fields: 'café' with '10.0' and 'gr', 'azúcar' with '10.0' and 'gr', and 'agua' with '50.0' and 'ml'. At the bottom of the window are two buttons: 'Guardar Cambios' and 'Cancelar'.

30. Método `_guardar_cambios_producto ()`:

Función: es el encargado de validar y guardar los cambios realizados a un producto.

Persistencia de datos:

- Guarda los cambios en el archivo JSON (`productos.json`) para que persistan después de cerrar el programa.

Datos importantes:

- **Validación estricta:** Asegura que no se guarden datos corruptos (ej: precios negativos).
- **Actualización en memoria y disco:** Los cambios se aplican tanto al objeto en memoria como al archivo JSON.

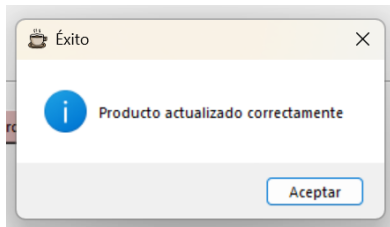
```
def _guardar_cambios_producto(self, nombre, precio, categoria):
    """Guarda los cambios del producto y vuelve a la lista"""
    try:
        # Validar datos
        if not nombre.strip():
            raise ValueError("El nombre no puede estar vacío")

        precio = float(precio)
        if precio <= 0:
            raise ValueError("El precio debe ser mayor que cero")

        # Actualizar producto
        self.producto_actual.nombre = nombre.strip()
        self.producto_actual.precio = precio
        self.producto_actual.categoria = categoria

        # Actualizar ingredientes
        for ingrediente, entry in self.ingredientes_entries.items():
            cantidad = float(entry.get())
            self.producto_actual.ingredientes[ingrediente] = cantidad

        # Guardar cambios
        self.guardar_productos()
        messagebox.showinfo("Éxito", "Producto actualizado correctamente")
        self.mostrar_lista_productos()
    except ValueError as e:
        messagebox.showerror("Error", f"Datos inválidos: {str(e)}")
```



Relación con otros métodos:

- Se llama desde el botón **"Guardar Cambios"** en el formulario de edición (`_mostrar_formulario_edicion`).
- Usa `self.guardar_productos()` para guardar en disco los cambios realizados.
- Vuelve a la vista de lista con `self.mostrar_lista_productos()`.

31. Método `_eliminar_producto()`:

Función: eliminar productos de la lista de manera segura.

Propósitos:

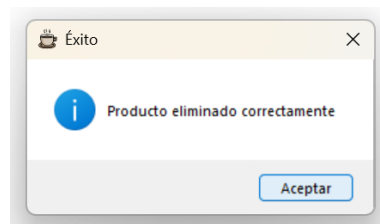
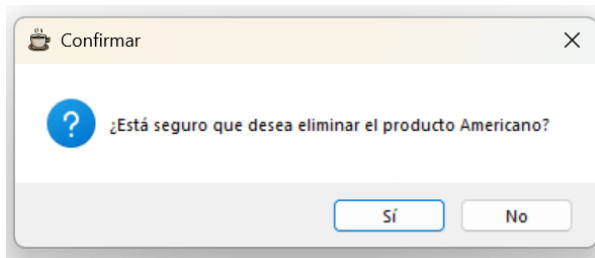
- **Eliminar productos:** Permite remover permanentemente un producto seleccionado (bebida o postre) del sistema.
- **Confirmación segura:** Solicita confirmación explícita del usuario antes de eliminar.
- **Actualización automática:**
 - Remueve el producto de la lista en memoria.

- Actualiza el archivo JSON (productos.json).
- Refresca la vista para mostrar la lista actualizada.

```
def _eliminar_producto(self, tree, productos):
    """Elimina el producto seleccionado"""
    seleccion = tree.selection()
    if not seleccion:
        messagebox.showwarning("Advertencia", "Por favor seleccione un producto para eliminar")
        return

    item = seleccion[0]
    idx = int(tree.index(item))
    producto = productos[idx]

    if messagebox.askyesno("Confirmar", f"¿Está seguro que desea eliminar el producto {producto.nombre}?"):
        productos.pop(idx)
        self.guardar_productos()
        messagebox.showinfo("Éxito", "Producto eliminado correctamente")
        self.mostrar_lista_productos()
```



Relación con otros métodos:

- Se llama desde:
 - El botón **"Eliminar Producto"** en la interfaz de gestión.
 - La tabla de productos (a través de eventos).
- Usa:
 - self.guardar_productos() para persistir los cambios.
 - self.mostrar_lista_productos() para refrescar la vista.

32. Método ver_inventario():

Función: mostrar una interfaz interactiva que permite administrar los ingredientes almacenados.

Propósitos:

- **Visualización del inventario:** Muestra todos los ingredientes existentes con sus cantidades y unidades de medida.
- **Gestión interactiva:** Permite:

- Actualizar cantidades de ingredientes
- Eliminar ingredientes del sistema
- Agregar nuevos ingredientes
- **Navegación cómoda:** Incluye scroll para manejar listas largas de ingredientes.

```
def ver_inventario(self):
    # Ocultar la ventana principal (administrador)
    self.root.withdraw()

    dialog = tk.Toplevel(self.root)
    dialog.title("Gestión de Inventario")
    dialog.geometry("600x600")

    # Función para manejar el cierre de la ventana
    def on_close():
        dialog.destroy()
        self.root.deiconify() # <--- Restaurar la ventana principal

    dialog.protocol("WM_DELETE_WINDOW", on_close) # <--- Configura acción al cerrar

    main_frame = tk.Frame(dialog)
    main_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    # Frame para agregar nuevo ingrediente
    add_frame = tk.Frame(main_frame)
    add_frame.pack(fill=tk.X, pady=(0, 10))

    tk.Button(
        add_frame,
        text="+ Agregar Nuevo Ingrediente",
        command=self.agregar_nuevo_ingrediente_dialog,
        bg="#4443627",
        fg="white",
        font=("Arial", 10, "bold")
    ).pack(fill=tk.X)

    # Frame para la lista de inventario
    list_frame = tk.Frame(main_frame)
    list_frame.pack(fill=tk.BOTH, expand=True)

    canvas = tk.Canvas(list_frame)
    scrollbar = ttk.Scrollbar(list_frame, orient="vertical", command=canvas.yview)
    scrollable_frame = tk.Frame(canvas)

    scrollable_frame.bind("<Configure>", lambda e: canvas.configure(scrollregion=canvas.bbox("all")))
    canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
    canvas.configure(yscrollcommand=scrollbar.set)

    # Encabezados
    tk.Label(scrollable_frame, text="Ingrediente", font=("Arial", 12, "bold")).grid(row=0, column=0, padx=5, pady=5, sticky="w")
    tk.Label(scrollable_frame, text="Cantidad", font=("Arial", 12, "bold")).grid(row=0, column=1, padx=5, pady=5)
    tk.Label(scrollable_frame, text="Unidad", font=("Arial", 12, "bold")).grid(row=0, column=2, padx=5, pady=5)
    tk.Label(scrollable_frame, text="Acciones", font=("Arial", 12, "bold")).grid(row=0, column=3, padx=5, pady=5)
```

```

# Filas de ingredientes
for i, (ingrediente, datos) in enumerate(self.inventario.obtener_estado().items(), 1):
    # Nombre del ingrediente
    tk.Label(scrollable_frame, text=ingrediente, font=("Arial", 10)).grid(row=i, column=0, padx=5, pady=2, sticky="w")

    # Cantidad (editable)
    cantidad_var = tk.StringVar(value=str(datos['cantidad']))
    cantidad_entry = tk.Entry(scrollable_frame, textvariable=cantidad_var, width=10, font=("Arial", 10))
    cantidad_entry.grid(row=i, column=1, padx=5, pady=2)

    # Unidad
    tk.Label(scrollable_frame, text=datos['unidad'], font=("Arial", 10)).grid(row=i, column=2, padx=5, pady=2)

    # Botones de acción
    action_frame = tk.Frame(scrollable_frame)
    action_frame.grid(row=i, column=3, padx=5, pady=2)

    tk.Button(
        action_frame,
        text="Actualizar",
        command=lambda ing=ingrediente, var=cantidad_var: self.actualizar_cantidad(ing, var),
        bg="#D8B585",
        fg="black",
        font=("Arial", 8, "bold"),
        width=8
    ).pack(side=tk.LEFT, padx=2)

    tk.Button(
        action_frame,
        text="Eliminar",
        command=lambda ing=ingrediente: self.eliminar_ingrediente(ing, dialog),
        bg="#F1E5D1",
        fg="black",
        font=("Arial", 8, "bold"),
        width=8
    ).pack(side=tk.LEFT, padx=2)

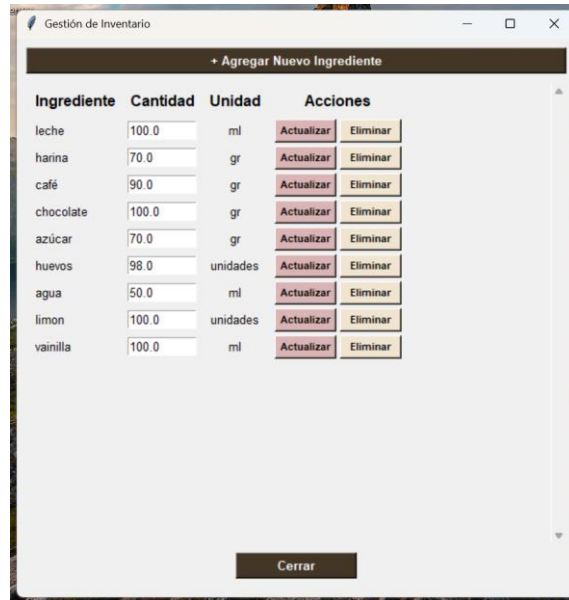
canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

tk.Button(
    main_frame,
    text="Cerrar",
    command=on_close,
    bg="#443627",
    fg="white",
    font=("Arial", 10, "bold"),
    width=15
).pack(pady=10)

```

Relación con otros componentes:

- **Depende de:**
 - self.inventario (objeto que maneja la lógica del inventario)
 - Métodos auxiliares como actualizar_cantidad() y eliminar_ingrediente()
- **Es llamado por:**
 - El menú principal de administración (botón "Ver Inventario")



23. Método `agregar_nuevo_ingrediente_dialog()`:

Función: mostrar un formulario emergente que permite registrar nuevos ingredientes en el inventario.

```
def agregar_nuevo_ingrediente_dialog(self):
    dialog = tk.Toplevel(self.root)
    dialog.title("Agregar Nuevo Ingrediente")
    dialog.geometry("300x200")

    tk.Label(dialog, text="Nombre:").grid(row=0, column=0, padx=5, pady=5, sticky="e")
    nombre_entry = tk.Entry(dialog)
    nombre_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Label(dialog, text="Cantidad:").grid(row=1, column=0, padx=5, pady=5, sticky="e")
    cantidad_entry = tk.Entry(dialog)
    cantidad_entry.grid(row=1, column=1, padx=5, pady=5)

    tk.Label(dialog, text="Unidad:").grid(row=2, column=0, padx=5, pady=5, sticky="e")
    unidad_var = tk.StringVar(value="gr")
    tk.OptionMenu(dialog, unidad_var, "gr", "ml", "unidades").grid(row=2, column=1, padx=5, pady=5, sticky="w")

    def confirmar():
        try:
            nombre = nombre_entry.get().strip()
            cantidad = float(cantidad_entry.get())
            unidad = unidad_var.get()

            if not nombre:
                raise ValueError("El nombre no puede estar vacío")

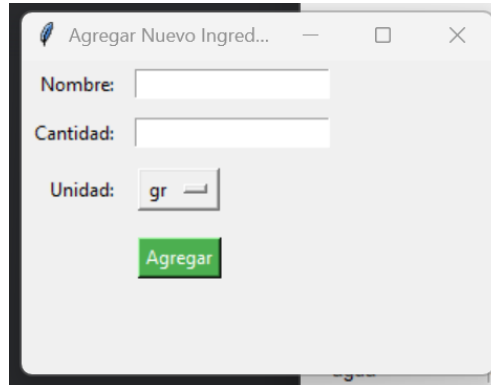
            self.inventario.agregar_nuevo_ingrediente(nombre, cantidad, unidad)
            messagebox.showinfo("Éxito", "Ingrediente agregado correctamente")
            dialog.destroy()
            self.ver_inventario() # Refrescar la vista
        except ValueError as e:
            messagebox.showerror("Error", f"Datos inválidos: {str(e)}")

    tk.Button(
        dialog,
        text="Agregar",
        command=confirmar,
        bg="#4CAF50",
        fg="white"
    ).grid(row=3, columnspan=2, pady=10)
```

Relación con otros componentes:

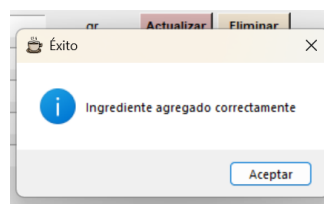
Dependencias:

- Requiere que `self.inventario` esté correctamente inicializado
- Asume la existencia de `ver_inventario()` para refrescar la vista



Confirmación:

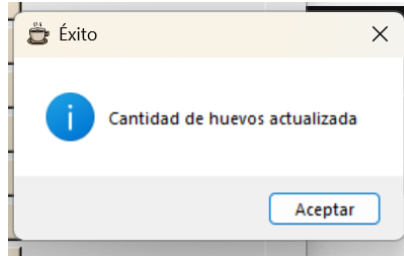
- Al hacer clic en "Agregar":
 - Valida los campos
 - Si es exitoso:
 - Cierra el diálogo (`dialog.destroy()`)
 - Actualiza la vista de inventario (`self.ver_inventario()`)



23. Método `actualizar_cantidad()`:

Función: actualizar la cantidad disponible de un ingrediente específico en el inventario del sistema, con validación de datos y retroalimentación visual.

```
def actualizar_cantidad(self, ingrediente, cantidad_var):
    try:
        nueva_cantidad = float(cantidad_var.get())
        self.inventario.actualizar_ingrediente(ingrediente, nueva_cantidad)
        messagebox.showinfo("Éxito", f"Cantidad de {ingrediente} actualizada")
    except ValueError as e:
        messagebox.showerror("Error", f"Dato inválido: {str(e)}")
```



23. Método eliminar_ingrediente():

Función: eliminar permanentemente un ingrediente específico del inventario del sistema, siguiendo un proceso seguro y confirmado por el usuario.

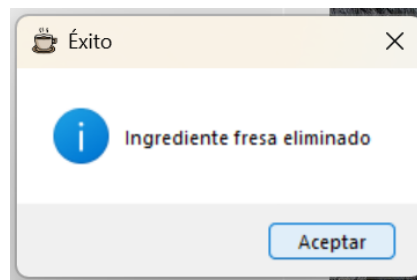
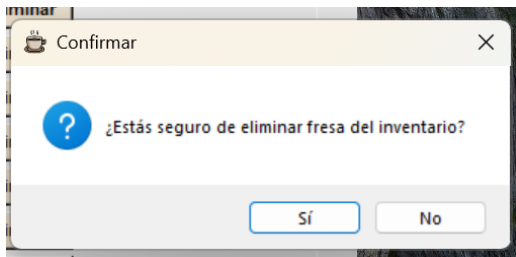
Eliminación propiamente dicha:

- Remueve el ingrediente del diccionario ingredientes del inventario
- Ejecuta guardar_inventario() para persistir el cambio en el archivo JSON

Manejo de resultados:

- **Éxito:** Cierra la ventana actual (parent_window) y refresca la vista del inventario
- **Error:** Detecta si el ingrediente no existe (captura KeyError)

```
def eliminar_ingrediente(self, ingrediente, parent_window):
    if messagebox.askyesno("Confirmar", f"¿Estás seguro de eliminar {ingrediente} del inventario?"):
        try:
            del self.inventario.ingredientes[ingrediente]
            self.inventario.guardar_inventario()
            messagebox.showinfo("Éxito", f"Ingrediente {ingrediente} eliminado")
            parent_window.destroy()
            self.ver_inventario() # Refrescar la vista
        except KeyError:
            messagebox.showerror("Error", f"No se encontró el ingrediente {ingrediente}")
```



24. Método `mostrar_historial_ventas()`:

Función: visualizar interactivamente todo el registro histórico de transacciones comerciales realizadas, permitiendo al administrador:

- Consultar ventas pasadas: Ver fecha, cliente, productos y montos
- Analizar patrones: Detectar productos más vendidos o clientes frecuentes
- Gestionar datos: Limpiar el historial completo o actualizar la vista

```
def mostrar_historial_ventas(self):
    # Oculta la ventana principal (administrador)
    self.root.withdraw() # <--- Oculta la ventana principal

    dialog = tk.Toplevel(self.root)
    dialog.title("Historial de Ventas")
    dialog.geometry("900x650")

    # Función para manejar el cierre de la ventana
    def on_close():
        dialog.destroy()
        self.root.deiconify() # <--- Restaura la ventana principal

    dialog.protocol("WM_DELETE_WINDOW", on_close) # <--- Configurar acción al cerrar

    # Frame principal
    main_frame = tk.Frame(dialog)
    main_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    # Frame para los botones de acción
    buttons_frame = tk.Frame(main_frame)
    buttons_frame.pack(fill=tk.X, pady=(0, 10))

    # Botón para limpiar historial
    tk.Button(buttons_frame,
              text="Limpiar Historial",
              command=self.limpiar_historial,
              bg="#DF5E5E",
              fg="white",
              font=("Arial", 10, "bold"),
              padx=15).pack(side=tk.LEFT, padx=5)

    # Botón para actualizar
    tk.Button(buttons_frame,
              text="Actualizar",
              command=lambda: self.actualizar_historial(tree),
              bg="#3C5B6F",
              fg="white",
              font=("Arial", 10, "bold"),
              padx=15).pack(side=tk.LEFT, padx=5)
```

```

# Botón para cerrar
tk.Button(buttons_frame,
          text="Cerrar",
          command=on_close,
          bg="#443627",
          fg="white",
          font=("Arial", 10, "bold"),
          padx=15).pack(side=tk.RIGHT, padx=5)

# Frame para el Treeview
tree_frame = tk.Frame(main_frame)
tree_frame.pack(fill=tk.BOTH, expand=True)

# Scrollbar
scrollbar = tk.Scrollbar(tree_frame)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

# Treeview para mostrar los pedidos
tree = ttk.Treeview(tree_frame,
                   columns=("Fecha", "Cliente", "Productos", "Total"),
                   yscrollcommand=scrollbar.set,
                   selectmode="extended")

# Configurar columnas
tree.heading("#0", text="ID", anchor=tk.W)
tree.heading("Fecha", text="Fecha", anchor=tk.W)
tree.heading("Cliente", text="Cliente", anchor=tk.W)
tree.heading("Productos", text="Productos", anchor=tk.W)
tree.heading("Total", text="Total", anchor=tk.W)

tree.column("#0", width=50, stretch=tk.NO)
tree.column("Fecha", width=150, stretch=tk.NO)
tree.column("Cliente", width=150)
tree.column("Productos", width=300)
tree.column("Total", width=100, stretch=tk.NO)

# Insertar datos
for i, pedido in enumerate(self.pedidos, 1):
    productos = ", ".join(pedido["productos"])
    tree.insert("", tk.END, text=str(i),
               values=(pedido["fecha"], pedido["cliente"], productos, f"${pedido['total']:.2f}"))

tree.pack(fill=tk.BOTH, expand=True)
scrollbar.config(command=tree.yview)

# Configurar estilo
style = ttk.Style()
style.configure("Treeview", rowheight=25, font=('Arial', 10))
style.configure("Treeview.Heading", font=('Arial', 10, 'bold'))

# Añadir doble clic para ver detalles
tree.bind("<Double-1>", lambda event: self.mostrar_detalle_pedido(tree, event))

```

ID	Fecha	Cliente	Productos	Total
1	2025-04-25 00:23:53	Fatima	Croissant	\$10.00
2	2025-04-25 02:00:09	Liz	Americano, Croissant	\$40.00

Interacción:

- **Doble clic:** Muestra detalles completos del pedido
- **Botones:** Ejecutan acciones específicas

Manejo de datos:

- Los datos vienen de self.pedidos (lista de diccionarios)
- Cada pedido tiene:

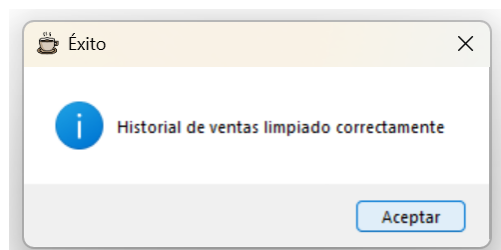
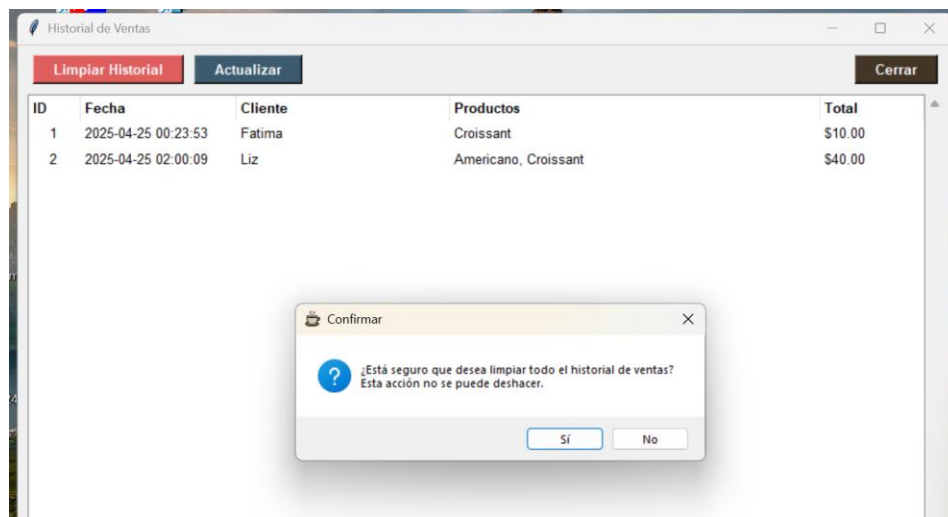
```
DESCARGAS Y COSAS > pedidos.json > ...
[
  {
    "cliente": "Fatima",
    "fecha": "2025-04-25 00:23:53",
    "productos": [
      "Croissant"
    ],
    "cantidades": [
      1
    ],
    "total": 10.0,
    "detalle": [
      "Croissant ($10.00)"
    ]
  },
  {
    "cliente": "Liz",
    "fecha": "2025-04-25 02:00:09",
    "productos": [
      "Americano",
      "Croissant"
    ]
  }
]
```

24. Método limpiar_historial():

Función: Eliminar permanentemente todo el registro histórico de ventas del sistema, proporcionando un mecanismo seguro y confirmado para resetear los datos de transacciones. Los cambios se escriben directamente al archivo JSON.

Antes	Después
Lista <code>pedidos</code> con N registros	Lista <code>pedidos</code> vacía []
Archivo JSON con historial	Archivo JSON con array vacío

```
def limpiar_historial(self):
    if messagebox.askyesno("Confirmar", "¿Está seguro que desea limpiar todo el historial de ventas?\nEsta acción no se puede deshacer."):
        self.pedidos = []
        try:
            self.guardar_pedidos()
            messagebox.showinfo("Éxito", "Historial de ventas limpiado correctamente")
        except Exception as e:
            messagebox.showerror("Error", f"No se pudo limpiar el historial: {str(e)}")
```



23. Método actualizar_historial ():

Función: Refrescar dinámicamente la tabla que muestra el historial de ventas, asegurando que los datos visibles coincidan exactamente con los registros actuales almacenados en el sistema.

Consideraciones importantes:

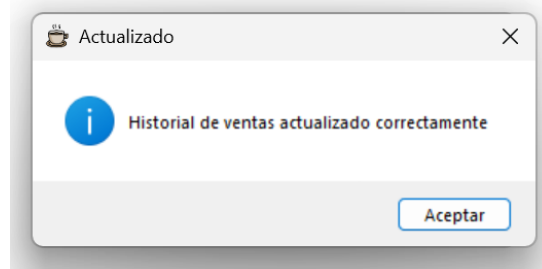
- Asume que self.pedidos contiene información correcta
- Puede causar "parpadeo" en interfaces con muchos registros
- Requiere que tree sea un Treeview válido y self.cargar_pedidos() funcione correctamente

```
def actualizar_historial(self, tree):
    # Limpiar el treeview
    for item in tree.get_children():
        tree.delete(item)

    # Recargar pedidos
    self.cargar_pedidos()

    # Insertar datos actualizados
    for i, pedido in enumerate(self.pedidos, 1):
        productos = ", ".join(pedido["productos"])
        tree.insert("", tk.END, text=str(i),
                    values=(pedido["fecha"], pedido["cliente"], productos, f"${pedido['total']:.2f}"))

    messagebox.showinfo("Actualizado", "Historial de ventas actualizado correctamente")
```



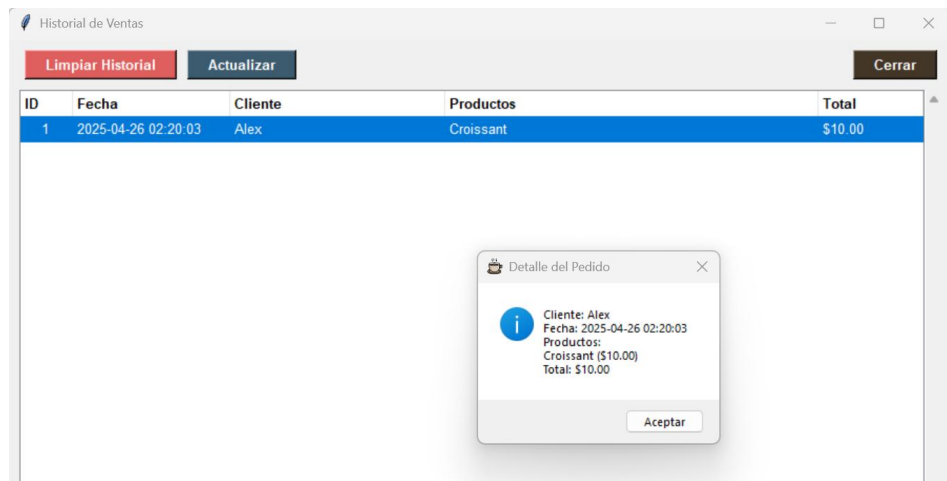
23. Método mostrar_detalle_pedido():

Función: Mostrar una vista ampliada con todos los detalles específicos de un pedido seleccionado en el historial de ventas, proporcionando más información que la que aparece en la tabla resumida.

```
def mostrar_detalle_pedido(self, tree, event):
    item = tree.selection()[0]
    valores = tree.item(item, 'values')

    # Buscar el pedido completo
    pedido = next((p for p in self.pedidos if p['fecha'] == valores[0] and p['cliente'] == valores[1]), None)

    if pedido:
        detalles = "\n".join(pedido["detalle"])
        messagebox.showinfo(
            "Detalle del Pedido",
            f"Cliente: {pedido['cliente']}\n"
            f"Fecha: {pedido['fecha']}\n"
            f"Productos: \n{detalles}\n"
            f"Total: ${pedido['total']:.2f}"
        )
    )
```



24. El bloque `if __name__ == "__main__":`:

Función: Iniciar la ejecución del programa y gestionar el ciclo de vida completo de la aplicación, desde la pantalla de inicio hasta el cierre.

Control de ejecución:

- Asegura que la aplicación se inicie correctamente
- Evita ejecuciones accidentales si el archivo se importa como módulo

Jerarquía visual:

- Establece la ventana raíz como contenedor padre de todos los demás elementos

Experiencia de usuario:

- Muestra primero una pantalla de bienvenida (SplashScreen)
- Luego carga el programa completo (CafeteriaApp)

```
if __name__ == "__main__":
    root = tk.Tk()
    splash = SplashScreen(root, CafeteriaApp)
    root.mainloop()
```

